

Parallelism in R Development with RcppParallel

A tutorial on parallel computing in R and C++ using RcppParallel, foreach, and biological sequence comparison as a motivating use case

datalab@unimib.it

Project:	dve-sample-r
Document:	Rcpp-HOWTO-Q6-all.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-r/notes/howtos/
Date:	February 19, 2026

Keywords: GEN, RcppParallel, Rcpp, OpenMP, foreach, parallelFor, parallelReduce, bioinformatics

Abstract

This tutorial introduces parallel programming in R and C++ within a CRAN-compliant package environment, using the RcppParallel library as the primary framework. The exposition targets R developers seeking to leverage multi-core hardware through both implicit parallelism (OpenMP, SIMD) and explicit task-based concurrency.

A biological sequence comparison problem — pairwise distance matrix computation on random DNA fragments using Hamming-based metrics — is used as the central motivating example. This use case provides a natural and pedagogically clear mapping to both parallelFor and parallelReduce primitives, avoiding artificial API demonstrations.

The C++ implementation follows the Google C++ Style Guide and exposes entry-point functions to R via Rcpp, with headers auto-generated by roxygen2 under inst/include. The accompanying R test script, located under exec/, uses foreach-based parallelism to execute benchmark trials across varying input sizes, aggregating timing and result metrics.

Core-allocation trade-offs between outer foreach workers and the inner RcppParallel thread pool are discussed, covering RcppParallel::setThreadOptions, doParallel::registerDoParallel, and parallel::makeCluster. Race condition analysis and synchronisation strategies are addressed, with preference for functional, message-passing patterns.

The tutorial also covers multi-node distribution strategies using ssh, MPI, and ZMQ, and provides a concise RcppParallel quick-start guide including DESCRIPTION and src/Makevars configuration, TinyThread vs. TBB backend selection, SIMD enablement via -march=native, and the renv installation workflow.

Contents

TOC	5
Q:1	6
Q:1 - Parallelism in R Development: A Tutorial	6
Role	6
Context	6
Objective	6
Specifications	7
C++ Code Requirements	7
R Script Requirements	7
CLI Arguments	7
test-script-logging-specification	8
Deliverables	8
Package Constraints	9
Output Format	9
A:1 (Claude)	10
Tutorial Narrative	10
1. C++ Implementation	10
src/dna_distance.h	10
src/dna_distance.cpp	12
2. R Test Script	14
exec/dna_benchmark.R	14
3. Concurrency Discussion	19
Thread safety in HammingMatrixWorker	19
Thread safety in MinPairReducer	19
Functional, message-passing character	19
Race-free crossover in R	19
Two-level thread pressure	19
4. Multi-Node Variant Discussion	20
SSH-based distribution	20
MPI-based distribution	20
ZMQ-based distribution	20
Data transport consideration	20
5. RcppParallel Quick-Start Guide	20
DESCRIPTION additions	20
src/Makevars	21
SIMD support in ~/.R/Makevars (local developer machine only)	21

OS-level dependencies (Ubuntu / Debian)	21
TinyThread vs. TBB backend	21
renv installation sequence	21
References	22
Additional Notes	22
A:1 (Gemini)	23
Task Overview	23
1. Tutorial Narrative: Parallel Sequence Analysis	23
Why RcppParallel?	23
The Benchmarking Harness	23
2. C++ Implementation	24
3. R Test Script	25
4. Concurrency Discussion	27
Race Conditions and Strategy	27
5. Multi-Node Variant Discussion	27
6. RcppParallel Quick Start Guide	28
Package Configuration	28
Installation Sequence	28
References /g	28
Additional Notes /g	29
A:1 (ChatGPT)	30
Overview /t	30
1. Tutorial Narrative	30
1.1 Problem Definition	30
1.2 Why This Fits RcppParallel	31
1.3 Nested Parallelism Strategy	31
2. C++ Implementation /t	31
Header Export	33
3. R Test Script (exec/benchmark-hamming.R)	34
4. Concurrency Discussion /t	36
4.1 Race Conditions	36
4.2 Oversubscription Risk	36
4.3 Core Allocation Strategies	37
5. Multi-Node Variant (Discussion)	37
5.1 SSH Clusters	37
5.2 MPI	37
5.3 ZeroMQ (ZMQ)	38
6. RcppParallel Quick Start Guide /t	38
6.1 DESCRIPTION	38
6.2 src/Makevars	38
6.3 Enabling SIMD Locally	38
6.4 Install Dependencies (Ubuntu)	39
6.5 Backend: TinyThread vs TBB	39

6.6 renv Workflow	39
7. References /t	39
Additional Notes /t	39
A:1 (Perplexity)	40
1. Tutorial Narrative /p	40
Parallel structure	40
2. C++ Implementation (Rcpp + RcppParallel)	41
src/dna_hamming.cpp	41
inst/include/dve-sample-r.h	45
3. R Test Script (exec/)	45
4. Concurrency Discussion /p	52
Natural parallel decomposition	52
Avoiding race conditions	52
Nested parallelism and oversubscription	53
5. Multi-Node Variant Discussion (No Code)	53
SSH-based orchestration	53
MPI-based orchestration	54
ZeroMQ (ZMQ) control and data transport	54
Serialization and data locality	54
6. RcppParallel Quick Start Guide /p	55
DESCRIPTION	55
src/Makevars	55
Local SIMD via ~/.R/Makevars	55
OS-level dependencies	55
TinyThread vs. TBB backend	56
renv sequence	56
References /p	56
Additional Notes /p	57
A:1 (DeepSeek)	58
Overview /d	58
Why This Example Fits RcppParallel	58
C++ Implementation	58
R Test Script	62
Concurrency Discussion	66
Race Conditions and Synchronization	66
Synchronization Strategy	66
Multi-Node Variant Discussion	67
Architecture Options	67
Work Distribution Strategy	67
Communication Patterns	67
RcppParallel Quick Start Guide	68
Package Configuration /d	68
OS Dependencies	68

Backend Selection	68
Testing Setup	69
References /d	69
Additional Notes	69
Performance Considerations	69
CRAN Compliance Tips	69
Extending the Example	69
A:1 (Mistral)	71
Overview /m	71
Tutorial Narrative /m	71
C++ Implementation /m	71
R Test Script /m	73
Concurrency Discussion /m	74
Multi-Node Variant Discussion /m	74
RcppParallel Quick Start Guide /m	74
References /m	75
Additional Notes /m	75

TOC

1. Q:1 - Parallelism in R Development: A Tutorial
 - see: [Parallel DNA Sequence Comparison with RcppParallel](#) (Claude)
 - see: [RcppParallel Tutorial: DNA Sequence Analysis](#) (Gemini)
 - see: [Parallel Biological Sequence Comparison in R and C++](#) (ChatGPT)
 - see: [RcppParallel Tutorial](#) (Perplexity)
 - see: [Parallel DNA Sequence Comparison with RcppParallel](#) (DeepSeek)
 - see: [Parallelism in R Development: A Tutorial](#) (Mistral)

Q:1

Q:1 - Parallelism in R Development: A Tutorial

□

Role

You are an expert R and C++ developer specialising in high-performance computing and parallel programming.

All code examples must be compact, clear, and focused on a well-defined subset of features from a single package. Code must be performant, using either implicit parallelism and vectorisation via OpenMP or SIMD intrinsics, or library-based interfaces to multitasking and multiprocessing OS facilities — selected to best fit the use case.

Context

- All examples will be integrated in a CRAN-compliant R package named `dve-sample-r`.
- The package is based on `renv` in *explicit* mode and already includes `Rcpp` and `RcppArmadillo`.

Objective

Prepare a structured tutorial introducing parallelism in R development, with examples written in both R and C++ using standard library support.

The tutorial must include a single, *inspiring* use-case example for the `RcppParallel` package, demonstrating both `parallelFor` and `parallelReduce` in a way that is *natural* to the algorithm — not a forced API demonstration. The chosen example must make meaningful use of both primitives.

Provide a simple implementation of a practical, inspiring use case:

- *"Biological Sequence Comparison (Smith-Waterman or Hamming-based distance matrix calculation) on random DNA fragments"*

This choice is motivated for its clear mapping to parallel reduction and map-reduce patterns.

The test R script must use `foreach` parallelism to run benchmark trials in parallel, collecting results for summary aggregation.

Different CPU-core allocation policies — distributing cores between the R `foreach` workers and the inner `RcppParallel` thread pool — must be discussed, including the relevant configuration functions

(RcppParallel::setThreadOptions, doParallel::registerDoParallel, parallel::makeCluster) and the trade-offs in total workload optimisation.

Specifications

C++ Code Requirements

- All C++ functions must be callable from R via Rcpp.
- Entry-point functions must be exportable as package C++ API, with headers auto-generated by roxygen2 placed under `inst/include`.
- Source files must reside under `src/`.
- Code must follow the *Google C++ Style Guide* and be moderately but clearly documented.
- Both `parallelFor` and `parallelReduce` from `RcppParallel` must be used; their roles must arise naturally from the algorithm structure.

R Script Requirements

- The test script must reside under `exec/` and be directly executable with a Rscript shebang line.
- `library()` calls must suppress warning messages.
- Command-line argument parsing must use a standard R argument-parsing library (e.g., `argparse` or `optparse`) and implement all arguments listed in the **CLI Arguments** section below.
- The script must report parsed arguments, execution trace, and detailed result dump with a logging channel, following specification described below, in heading:
 - **test-script-logging-specification**

CLI Arguments

Generic Arguments

- `help`: option `-h/--help`, boolean, default `FALSE` — prints usage and exits without execution.
- `verbose`: option `-v/--verbose`, boolean, default `FALSE` — sets logging level (`FALSE` = info, `TRUE` = debug).
- `rnd_seed`: option `-u/--seed`, integer, default `0` — initialises the random number generator for deterministic output.

Common Arguments

- `save_data`: option `-s/--save`, boolean — when set, dumps random input data and results to an external file (text or JSON) for further analysis or plotting.
- `input_size`: positional, multiple values — specifies the cardinality of the input domain (e.g. DNA fragment length).

Algorithm-Specific Arguments

Include additional arguments with sensible defaults as required by the chosen example. Examples:

- `graph_density`: option `-g/--density` — ratio of edges to nodes (`0.0` = fully isolated, `1.0` = fully connected).

- `arrival_rate`: option `-r/--rate` — traffic arrival rate for toll simulation.
- `sequence_similarity`: option `-c/--cross` — number of random crossover events between sequences.

test-script-logging-specification

- Script output should go to stdout and be logged to a file using standard logger facilities.
 - Log message formatting can use `glue` package facilities.
 - The log directory will also be used for storing benchmark results and plots.
 - The log directory path will be taken from the environment variable `P_LOGS_DIR`, with `logs` as the default.
 - The log directory should be created if it does not exist.
 - The log filename should have the prefix `<script-name>-<sec-timestamp>` with a `.log` extension.
 - The `<sec-timestamp>` part is composed of the script's start time, formatted as localtime in `"YYYYMMDD-hhmmss"` format.
 - The script's preparation and execution phases should be logged at the info level (arguments, benchmark invocation, final summary), while the final report section should be logged at the `"debug"` level (`verbose >= 1`).
 - All log artifacts should contain the test type and a localtime timestamp suffix as part of the filename.
 - During script initialization, log: 1. the script arguments, 2. the full path of the log directory, and 3. the output of the system command: `inxi -C`.
-

Deliverables

Deliver the following, in order:

1. **Tutorial Narrative** — Introduce the chosen algorithm, explain why it is a good fit for `RcppParallel`, and explain how `foreach/parallelly` is used in the test harness.
2. **C++ Implementation** — Parallel implementation of the chosen example using `parallelFor` and `parallelReduce`. Include concise inline comments.
3. **R Test Script** — Driver script under `exec/` that calls the C++ functions for varying input sizes, collects elapsed times and result metrics, and runs trials in parallel using `foreach`.
4. **Concurrency Discussion** — Identify potential race conditions in the implementation and document the synchronisation strategy used. Prefer a functional, message-passing approach; use mutexes only around non-blocking critical sections when strictly necessary.
5. **Multi-Node Variant Discussion** (*discussion only — no code required*) — Describe the R script modifications needed to distribute work across a multi-node cluster, with a focus on `ssh`, `MPI`, and `ZMQ` for control and data transport.
6. **RcppParallel Quick Start Guide** — Describe the minimum package configuration steps:

- Required modifications to DESCRIPTION and src/Makevars.
- A note on enabling SIMD support via `-march=native` in `~/R/Makevars`, including the expected R CMD check warning and why it is acceptable locally.
- Any required OS-level library dependencies, with `apt install` commands where applicable.
- Notes on TinyThread vs. TBB backend selection.
- The `renv` installation sequence: `renv::install()` followed by `renv::snapshot()`.

Package Constraints

- The package must be CRAN-compliant and pass R CMD check.
- The only admitted warning is the one produced by `-march=native` in a local `~/R/Makevars` — this must *not* appear in `src/Makevars`.
- Follow tidyverse style conventions for all R code.

Output Format

- Reply in well-formed *GitLab Flavoured Markdown (GLFM)* with strict lint compliance:
 - Fenced code block delimiters (`` ```) must begin at column 0; do not nest code blocks.
 - Use `_underscore_` markup for emphasis.
 - Prefer nested headings over bold text for structure.
 - Use only - (dash) for unordered lists, with correct indentation.
 - Insert a blank line after every heading, list block, and code block.
- At the end of the document, provide a `## References` section with Markdown footnotes (prefixed `rf-`) linked inline to relevant passages, covering package documentation and algorithm references.
- Add an `## Additional Notes` section for any important information not explicitly required above.
- Ignore all rendering-pipeline markup present in this prompt: `<details><summary>` HTML blocks, `{=latex}` code blocks, `[!tip]/[!note]` block quotes, code-folding tags (three braces pairs), and internal cross-reference links.

A:1 (Claude)

▣ *Parallel DNA Sequence Comparison with RcppParallel*

Tutorial Narrative

DNA sequence analysis is a canonical workload in bioinformatics: given N short nucleotide fragments, compute a pairwise distance matrix of dimension $N \times N$. Two natural primitives emerge immediately from the problem structure:

- **parallelFor** — each row i of the distance matrix is independent; rows can be computed concurrently without any shared mutable state.
- **parallelReduce** — the minimum edit score across all pairs (the "closest pair" statistic) is a classic reduction: commutative, associative, and embarrassingly splittable.

We use a simplified Hamming distance (mismatches per aligned position) rather than full Smith-Waterman to keep the tutorial focused on the parallel primitives rather than the DP recurrence. The algorithm maps cleanly: filling the upper triangle of the matrix is a **parallelFor** over rows; finding the globally most-similar pair is a **parallelReduce** over the filled matrix.

The R test harness uses `foreach` with a `doParallel` backend to launch multiple independent benchmark *trials* (varying input sizes or repetitions) as outer parallel workers. Each worker calls into the C++ layer, which spawns its own `RcppParallel` thread pool. This two-level hierarchy is the central tension discussed in the concurrency section.

1. C++ Implementation

src/dna_distance.h

```
// dna_distance.h — Exported C++ API header (auto-placed under inst/include
// by roxygen2 LinkingTo mechanism).
//
// Declares the parallel Hamming distance matrix worker and the parallel
// minimum-pair reduction worker, both callable from R via Rcpp.

#pragma once

#include <RcppArmadillo.h>
#include <RcppParallel.h>
```

```

#include <string>
#include <vector>

namespace dve {

// Fills the upper triangle of a symmetric N×N distance matrix in parallel.
// Each element [i,j] holds the normalised Hamming distance in [0,1].
struct HammingMatrixWorker : public RcppParallel::Worker {
    const std::vector<std::string>& seqs; // input sequences (read-only)
    RcppParallel::RMatrix<double> mat;    // output matrix (row-parallel writes)
    const std::size_t len;               // sequence length (uniform)

    HammingMatrixWorker(const std::vector<std::string>& seqs,
                        Rcpp::NumericMatrix mat, std::size_t len)
        : seqs(seqs), mat(mat), len(len) {}

    // Called by the thread pool: computes all [i, j>i] distances for row i.
    void operator()(std::size_t begin, std::size_t end) override;
};

// Finds the pair (i,j) with the minimum distance via parallel reduction.
struct MinPairReducer : public RcppParallel::Reducer {
    const RcppParallel::RMatrix<double>& mat;
    double min_dist;
    int min_i, min_j;

    explicit MinPairReducer(const RcppParallel::RMatrix<double>& mat)
        : mat(mat), min_dist(1.0 + 1e-9), min_i(-1), min_j(-1) {}

    // Split constructor required by Intel TBB / TinyThread split protocol.
    MinPairReducer(const MinPairReducer& other, RcppParallel::Split)
        : mat(other.mat), min_dist(1.0 + 1e-9), min_i(-1), min_j(-1) {}

    void operator()(std::size_t begin, std::size_t end) override;

    // Merge sibling reducer results; keep the globally smaller distance.
    void join(const MinPairReducer& rhs) {
        if (rhs.min_dist < min_dist) {
            min_dist = rhs.min_dist;
            min_i     = rhs.min_i;
            min_j     = rhs.min_j;
        }
    }
};
};

```

```
} // namespace dve
```

src/dna_distance.cpp

```
// dna_distance.cpp - Parallel Hamming distance matrix and minimum-pair
// reduction, implemented with RcppParallel primitives.
//
// Google C++ Style Guide; clang-format width 88.

#include "dna_distance.h"
#include <Rcpp.h>
#include <RcppParallel.h>
#include <algorithm>
#include <cstdint>
#include <string>
#include <vector>

using namespace Rcpp;
using namespace RcppParallel;

// -----
// HammingMatrixWorker::operator()
// -----
// Runs in a thread assigned the half-open row range [begin, end).
// Writes only to columns j > i in row i - no two threads share a cell.
void dve::HammingMatrixWorker::operator()(std::size_t begin,
                                           std::size_t end) {
    const std::size_t n = seqs.size();
    for (std::size_t i = begin; i < end; ++i) {
        for (std::size_t j = i + 1; j < n; ++j) {
            std::size_t mismatches = 0;
            const auto& a = seqs[i];
            const auto& b = seqs[j];
            // Inner loop is auto-vectorisable (SIMD) when compiled with -O2.
            for (std::size_t k = 0; k < len; ++k) {
                mismatches += (a[k] != b[k]) ? 1u : 0u;
            }
            const double d = static_cast<double>(mismatches) / static_cast<double>(len);
            mat(i, j) = d;
            mat(j, i) = d; // symmetric fill; j-row write is safe - j > i, unique cell
        }
    }
}

// -----
```

```

// MinPairReducer::operator()
// -----
// Scans the upper triangle of rows [begin, end) looking for a new minimum.
void dve::MinPairReducer::operator()(std::size_t begin, std::size_t end) {
    const std::size_t n = mat.nrow();
    for (std::size_t i = begin; i < end; ++i) {
        for (std::size_t j = i + 1; j < n; ++j) {
            const double d = mat(i, j);
            if (d < min_dist) {
                min_dist = d;
                min_i      = static_cast<int>(i);
                min_j      = static_cast<int>(j);
            }
        }
    }
}

// -----
// R-callable entry points
// -----

//' Compute a pairwise Hamming distance matrix for DNA sequences in parallel.
//'
//' @param sequences A character vector of equal-length DNA strings.
//' @param num_threads Number of RcppParallel worker threads (0 = auto).
//' @return A symmetric numeric matrix of normalised Hamming distances.
//' @export
// [[Rcpp::export]]
NumericMatrix hamming_distance_matrix(const std::vector<std::string>& sequences,
                                      int num_threads = 0) {
    const std::size_t n    = sequences.size();
    const std::size_t len = sequences.empty() ? 0u : sequences[0].size();

    if (num_threads > 0) {
        RcppParallel::setThreadOptions(num_threads);
    }

    NumericMatrix mat(n, n); // zero-initialised; diagonal stays 0.0

    dve::HammingMatrixWorker worker(sequences, mat, len);
    RcppParallel::parallelFor(0, n, worker);

    return mat;
}

```

```

//' Find the most-similar sequence pair via parallel reduction.
//'
//' Operates on the matrix produced by \code{hamming_distance_matrix}.
//'
//' @param mat A symmetric numeric distance matrix.
//' @return A named list: \code{dist}, \code{i}, \code{j} (1-based indices).
//' @export
// [[Rcpp::export]]
List find_closest_pair(NumericMatrix mat) {
  RMatrix<double> rmat(mat);
  dve::MinPairReducer reducer(rmat);
  RcppParallel::parallelReduce(0, static_cast<std::size_t>(mat.nrow()), reducer);

  return List::create(
    Named("dist") = reducer.min_dist,
    Named("i")    = reducer.min_i + 1,    // convert to 1-based
    Named("j")    = reducer.min_j + 1);
}

```

2. R Test Script

exec/dna_benchmark.R

```

#!/usr/bin/env Rscript
# exec/dna_benchmark.R - Benchmark driver for parallel Hamming distance matrix.
#
# Runs benchmark trials across input sizes using foreach outer parallelism;
# each trial calls into the RcppParallel C++ layer.
#
# Usage:
#   Rscript exec/dna_benchmark.R [options] <input_size> [<input_size> ...]

# — Package loading —
suppressWarnings({
  library(argparse)
  library(logger)
  library(glue)
  library(foreach)
  library(doParallel)
  library(parallel)
  library(RcppParallel)
  library(dve.sample.r) # the host package
})

```

```

# — Script metadata —
script_name <- "dna_benchmark"
start_time  <- Sys.time()
ts_suffix   <- format(start_time, "%Y%m%d-%H%M%S")

# — Log directory —
log_dir <- Sys.getenv("P_LOGS_DIR", unset = "logs")
if (!dir.exists(log_dir)) dir.create(log_dir, recursive = TRUE)
log_file <- file.path(log_dir, glue("{script_name}-{ts_suffix}.log"))

log_appender(appender_tee(log_file)) # stdout + file
log_threshold(INFO)                  # default; raised to DEBUG if --verbose

# — Argument parsing —
parser <- ArgumentParser(description = "Parallel Hamming distance benchmark")

parser$add_argument("-v", "--verbose",
  action = "store_true", default = FALSE,
  help   = "Enable debug-level logging [default: FALSE]")

parser$add_argument("-u", "--seed",
  type     = "integer", default = 0L, dest = "rnd_seed",
  help     = "RNG seed for reproducibility [default: 0]")

parser$add_argument("-s", "--save",
  action = "store_true", default = FALSE, dest = "save_data",
  help   = "Dump input data and results to JSON")

parser$add_argument("-n", "--num-sequences",
  type     = "integer", default = 200L, dest = "num_sequences",
  help     = "Number of DNA sequences per trial [default: 200]")

parser$add_argument("-c", "--cross",
  type     = "integer", default = 0L, dest = "sequence_similarity",
  help     = "Number of crossover events between sequences [default: 0]")

parser$add_argument("-w", "--foreach-workers",
  type     = "integer", default = 2L, dest = "foreach_workers",
  help     = "Outer foreach parallel workers [default: 2]")

parser$add_argument("-t", "--rcpp-threads",
  type     = "integer", default = 2L, dest = "rcpp_threads",
  help     = "Inner RcppParallel thread count per worker [default: 2]")

```

```

parser$add_argument("input_size",
  type = "integer", nargs = "+",
  help = "One or more sequence lengths (positional)")

args <- parser$parse_args()

# — Apply verbose flag —————
if (args$verbose) log_threshold(DEBUG)

# — Log initialisation block —————
log_info(glue("Script      : {script_name}"))
log_info(glue("Log directory : {normalizePath(log_dir)}"))
log_info(glue("Parsed args   : seed={args$rnd_seed}, sizes={toString(args$input_size)}, \\
  foreach_workers={args$foreach_workers}, rcpp_threads={args$rcpp_threads}, \\
  num_sequences={args$num_sequences}, crossovers={args$sequence_similarity}"))

# System CPU topology
cpu_info <- tryCatch(system("inxi -C", intern = TRUE), error = function(e) "(inxi not available)")
log_info(glue("System CPU:\n{paste(cpu_info, collapse = '\n')}"))

# — Helpers —————

#' Generate a random DNA sequence of a given length.
random_dna <- function(len, seed = NULL) {
  if (!is.null(seed)) set.seed(seed)
  paste(sample(c("A", "C", "G", "T"), len, replace = TRUE), collapse = "")
}

#' Apply crossover events between two sequences (single-point swap).
crossover <- function(a, b, n_events) {
  if (n_events == 0L) return(a)
  len <- nchar(a)
  chars <- strsplit(a, "")[[1L]]
  bchars <- strsplit(b, "")[[1L]]
  for (i in seq_len(n_events)) {
    pos <- sample.int(len - 1L, 1L)
    tail_a <- chars[(pos + 1L):len]
    chars[(pos + 1L):len] <- bchars[(pos + 1L):len]
    bchars[(pos + 1L):len] <- tail_a
  }
  paste(chars, collapse = "")
}

#' Generate a panel of DNA sequences with optional crossovers.
make_sequences <- function(n, len, n_cross, seed) {

```

```

set.seed(seed)
seqs <- replicate(n, random_dna(len))
if (n_cross > 0L && n ≥ 2L) {
  seqs[1L] <- crossover(seqs[1L], seqs[2L], n_cross)
}
seqs
}

# — Parallel setup —————
total_cores <- parallel::detectCores(logical = FALSE)
foreach_workers <- min(args$foreach_workers, total_cores)
rcpp_threads <- args$rcpp_threads

log_info(glue("Physical cores detected : {total_cores}"))
log_info(glue("foreach workers          : {foreach_workers}"))
log_info(glue("RcppParallel threads    : {rcpp_threads} (per worker)"))
log_info(glue("Total thread pressure   : {foreach_workers * rcpp_threads} \\
              (ideal ≤ {total_cores})"))

cl <- parallel::makeCluster(foreach_workers, type = "PSOCK")
doParallel::registerDoParallel(cl)

# Export variables required inside foreach workers.
parallel::clusterExport(cl, varlist = c("make_sequences", "random_dna",
                                         "crossover", "rcpp_threads"),
                        envir = environment())
parallel::clusterEvalQ(cl, {
  suppressWarnings(library(RcppParallel))
  suppressWarnings(library(dve.sample.r))
  RcppParallel::setThreadOptions(rcpp_threads)
})

# — Benchmark trials —————
log_info("Starting benchmark trials ...")

trial_grid <- expand.grid(
  seq_len = args$input_size,
  trial   = seq_len(3L)    # three repetitions per size for stable timing
)

results <- foreach(
  row      = iterators::iter(trial_grid, by = "row"),
  .combine = rbind,
  .packages = c("dve.sample.r", "RcppParallel")
) %dopar% {

```

```

seq_len_val <- row$seq_len
trial_idx   <- row$trial
seed_val    <- args$rnd_seed + trial_idx

seqs <- make_sequences(args$num_sequences, seq_len_val,
                       args$sequence_similarity, seed_val)

t0 <- proc.time()[["elapsed"]]
mat <- hamming_distance_matrix(seqs, num_threads = rcpp_threads)
t1 <- proc.time()[["elapsed"]]

closest <- find_closest_pair(mat)

data.frame(
  seq_len    = seq_len_val,
  trial      = trial_idx,
  elapsed_s  = t1 - t0,
  min_dist   = closest$dist,
  pair_i     = closest$i,
  pair_j     = closest$j,
  stringsAsFactors = FALSE
)
}

parallel::stopCluster(cl)

# — Summary —————
summary_tbl <- aggregate(elapsed_s ~ seq_len, data = results, FUN = mean)
names(summary_tbl)[2L] <- "mean_elapsed_s"

log_info("≡≡ Benchmark Summary ≡≡")
for (i in seq_len(nrow(summary_tbl))) {
  log_info(glue(" seq_len={summary_tbl$seq_len[i]} \\
               mean_elapsed={round(summary_tbl$mean_elapsed_s[i], 4)}s"))
}

log_debug("≡≡ Full Results ≡≡")
for (i in seq_len(nrow(results))) {
  r <- results[i, ]
  log_debug(glue(" seq_len={r$seq_len} trial={r$trial} elapsed={round(r$elapsed_s,4)}s \\
               min_dist={round(r$min_dist,4)} pair=({r$pair_i},{r$pair_j})"))
}

# — Optional data dump —————
if (args$save_data) {

```

```

out_path <- file.path(log_dir, glue("{script_name}-{ts_suffix}-results.json"))
jsonlite::write_json(results, out_path, pretty = TRUE)
log_info(glue("Results saved to: {out_path}"))
}

log_info("Done.")

```

3. Concurrency Discussion

Thread safety in HammingMatrixWorker

The matrix fill is safe without any mutex because the work partition is *row-disjoint*: `parallelFor` assigns non-overlapping row ranges `[begin, end)` to each thread. Thread t writes to cells (i, j) and (j, i) for all $j > i$ in its assigned rows. The symmetric write `mat(j, i)` writes into row j , which may be outside thread t 's *assigned* range — however, it writes into a cell that no other thread will ever write, because the only thread that would write `mat(j, i)` via the upper-triangle loop would be thread t' assigned row j , and that thread only processes $k > j$, never $k = i < j$. The write pattern is therefore guaranteed non-overlapping across threads, and no synchronisation is required.

`RMatrix<double>` (the `RcppParallel` wrapper) prevents accidental R GC entanglement; direct `double*` arithmetic is used internally.

Thread safety in MinPairReducer

`parallelReduce` guarantees that each reducer instance is accessed by exactly one thread at a time. The `join()` method is called sequentially by the runtime to merge sibling subtask results; it never races with `operator()`. Local minimum tracking within `operator()` is purely stack-local and requires no synchronisation.

Functional, message-passing character

Both workers are *value types* that own their local state and communicate results only through the structured `join()` protocol mandated by the `RcppParallel` API. This is effectively message-passing without explicit channels: the runtime acts as the message bus.

Race-free crossover in R

The R-side `crossover()` helper operates on local copies of character vectors; the `make_sequences()` function is called inside each `foreach` worker in a separate R process (PSOCK cluster), so there is no shared mutable state at the R level either.

Two-level thread pressure

The principal operational risk is *thread over-subscription*: if `foreach_workers × rcpp_threads > physical_cores`, the OS will time-slice threads, causing cache-line contention and context-switch

overhead. The logged warning line (Total thread pressure: X (ideal $\leq Y$)) makes this visible at runtime. A safe default for P physical cores is `foreach_workers = 2, rcpp_threads = P/2`.

4. Multi-Node Variant Discussion

SSH-based distribution

Replace `parallel::makeCluster(N, type = "PSOCK")` with `parallel::makeCluster(c("node1", "node2", ...), type = "PSOCK")`. R spawns worker processes over SSH using `Rscript`; the package must be installed on every node. Network latency makes this suitable only when trial granularity is large (sequence lengths $> 10\,000$, many sequences).

MPI-based distribution

Use `Rmpi` with `doMPI::registerDoMPI`. Launch the script with `mpirun -np W Rscript exec/dna_benchmark.R`. ... MPI provides low-latency collective operations and is the right choice when the result matrix itself must be aggregated across nodes (e.g. distributed distance matrix rows). Data serialisation uses R's native serialisation or `pbdMPI` raw-buffer transfers.

ZMQ-based distribution

`pbdZMQ` or the `clustermq` package allow a push/pull task queue pattern. A single coordinator process pushes trial descriptors onto a PUSH socket; worker processes on remote nodes pull tasks, execute trials, and PUSH results back to a PULL collector. This is the most fault-tolerant topology (workers can join and leave dynamically) and avoids the SSH-per-worker connection setup of PSOCK clusters. `clustermq` with a `scheduler = "slurm"` backend requires only changing one `registerDoParallel` call.

Data transport consideration

In all multi-node cases the DNA sequences (character vectors) must be serialised and transmitted. For large N and long sequences, pre-generating sequences on each worker (using the shared RNG seed) avoids transmitting the input entirely — only the scalar trial descriptor (size, seed, crossover count) need travel over the network.

5. RcppParallel Quick-Start Guide

DESCRIPTION additions

```
Imports:
  RcppParallel
LinkingTo:
  Rcpp,
```

```
RcppArmadillo,
RcppParallel
SystemRequirements: GNU make
```

src/Makevars

```
## Link against the RcppParallel thread library (TinyThread or TBB).
PKG_LIBS += $(shell "${R_HOME}/bin/Rscript" -e "RcppParallel::RcppParallelLibs()")
PKG_CXXFLAGS += $(shell "${R_HOME}/bin/Rscript" -e "RcppParallel::CxxFlags()")
```

Do *not* add `-march=native` here; it produces a R CMD check NOTE/WARNING that blocks CRAN submission.

SIMD support in `~/R/Makevars` (local developer machine only)

```
CXXFLAGS = -O2 -march=native
```

This flag allows the compiler to emit AVX2 / AVX-512 vectorised code for the inner Hamming loop. R CMD check `--as-cran` will warn that the binary is not portable; this is expected and acceptable for local development and benchmarking. Never commit this to `src/Makevars`.

OS-level dependencies (Ubuntu / Debian)

```
# TinyThread is header-only and bundled with RcppParallel – no extra apt package.
# For TBB backend (optional, higher throughput on many-core systems):
sudo apt install libtbb-dev
```

TinyThread vs. TBB backend

Property	TinyThread (default)	Intel TBB
Installation	bundled, zero-config	libtbb-dev required
Scheduler	static work-stealing	dynamic work-stealing
NUMA-aware	no	yes (TBB 2021+)
Best for	≤ 8 cores, portable	≥ 16 cores, server workloads

Select TBB at runtime with `RcppParallel::setThreadOptions(backend = "tbb")` or at compile time by defining `RCPP_PARALLEL_USE_TBB=1`.

renv installation sequence

```
# Run from the package root with renv in explicit mode.
renv::install("RcppParallel")
renv::install("doParallel")
renv::install("foreach")
renv::install("iterators")
renv::install("logger")
```

```
renv::install("argparse")
renv::install("glue")
renv::install("jsonlite")
renv::snapshot()
```

References

Additional Notes

- The symmetric write pattern in `HammingMatrixWorker` (writing `mat(j, i)` from thread assigned row `i`) is safe but subtle. If future maintainers change the partitioning scheme, they must re-verify this invariant. A conservative alternative is to fill only the upper triangle and mirror with a second sequential pass.
- The `iterators` package must be explicitly installed (`renv::install("iterators")`) because `foreach` lists it as a dependency but `renv` explicit mode does not resolve transitive dependencies automatically.
- `argparse` requires Python 3 with the `argparse` module available on `PATH`. On minimal CI images, substitute `optparse` and adjust the parser calls accordingly.
- When `--save` is used, `jsonlite` must also be installed (`renv::install("jsonlite")`).
- `PSOCK` clusters on Windows do not support `clusterEvalQ` before `registerDoParallel`; the `RcppParallel::setThreadOptions` call inside `clusterEvalQ` must be moved into the `foreach` body for Windows compatibility.
- For CRAN submission, remove the `exec/` script from the package tarball or list it under `.Rbuildignore`; CRAN policy does not require it but executable scripts without a documented interface can trigger manual review.

A:1 (Gemini)

▢ *RcppParallel Tutorial: DNA Sequence Analysis*

Task Overview

This tutorial provides a foundational guide to high-performance parallel computing within the R ecosystem, specifically focusing on the integration of `RcppParallel` for fine-grained multi-threading and `foreach` for coarse-grained task distribution.

1. Tutorial Narrative: Parallel Sequence Analysis

In bioinformatics, comparing large sets of DNA sequences is a computationally intensive task. When calculating a distance matrix (e.g., Hamming distance) for sequences, we perform comparisons. This is a classic "embarrassingly parallel" problem.

Why RcppParallel?

While R provides high-level parallel functions, `RcppParallel` allows us to move the heavy lifting to C++ using Intel TBB (Threading Building Blocks). We utilize two primary patterns:

- **parallelFor**: Maps the distance calculation across the matrix rows. Each thread handles a subset of the sequences, ensuring all CPU cores are saturated.
- **parallelReduce**: Aggregates the results. After computing distances, we often need a global metric, such as the "Total Genetic Divergence." Instead of using a global variable with a mutex (which causes contention), `parallelReduce` allows each thread to sum its local results and then merge them efficiently.

The Benchmarking Harness

To optimize a package, we must test different concurrency strategies. We use `foreach` at the R level to run multiple benchmark "trials" (e.g., different DNA lengths or sequence counts). This creates a two-tier parallel structure: the R worker manages the trial, while the C++ code manages the internal CPU threads.

2. C++ Implementation

The following code implements the DNA distance matrix calculation. It is designed to be placed in `src/dna_distance.cpp`.

```
#include <RcppArmadillo.h>
#include <RcppParallel.h>
#include <string>
#include <vector>
#include <algorithm>

// [[Rcpp::depends(RcppArmadillo, RcppParallel)]]

using namespace Rcpp;
using namespace RcppParallel;

// Worker for calculating Hamming Distance Matrix
struct DistanceWorker : public Worker {
    const std::vector<std::string> sequences;
    RMatrix<double> output;

    DistanceWorker(const std::vector<std::string> seqs, NumericMatrix out)
        : sequences(seqs), output(out) {}

    void operator()(std::size_t begin, std::size_t end) {
        for (std::size_t i = begin; i < end; ++i) {
            for (std::size_t j = i + 1; j < sequences.size(); ++j) {
                double dist = 0;
                std::size_t len = std::min(sequences[i].length(), sequences[j].length());
                for (std::size_t k = 0; k < len; ++k) {
                    if (sequences[i][k] != sequences[j][k]) dist++;
                }
                output(i, j) = dist;
                output(j, i) = dist; // Symmetric matrix
            }
        }
    }
};

// Reducer for calculating Total Genetic Divergence
struct DivergenceReducer : public Worker {
    const RMatrix<double> input;
    double value;

    DivergenceReducer(const NumericMatrix in) : input(in), value(0) {}
```

```

DivergenceReducer(const DivergenceReducer& split, Split)
: input(split.input), value(0) {}

void operator()(std::size_t begin, std::size_t end) {
    for (std::size_t i = begin; i < end; ++i) {
        for (std::size_t j = i + 1; j < input.ncol(); ++j) {
            value += input(i, j);
        }
    }
}

void join(const DivergenceReducer& rhs) { value += rhs.value; }
};

// Calculate DNA Distance Matrix
// @export
// [[Rcpp::export]]
List compute_dna_metrics(CharacterVector seqs) {
    int n = seqs.size();
    std::vector<std::string> std_seqs = as<std::vector<std::string>>(seqs);
    NumericMatrix dist_mat(n, n);

    // Parallel compute matrix
    DistanceWorker dist_worker(std_seqs, dist_mat);
    parallelFor(0, n, dist_worker);

    // Parallel reduction for total divergence
    DivergenceReducer div_reducer(dist_mat);
    parallelReduce(0, n, div_reducer);

    return List::create(_["matrix"] = dist_mat,
                        _["total_divergence"] = div_reducer.value);
}

```

3. R Test Script

This script is located at `exec/bench_dna.R`.

```
#!/usr/bin/env Rscript
```

```
suppressMessages({
  library(argparse)
  library(logger)

```

```

library(glue)
library(foreach)
library(doParallel)
library(dve.sample.r)
})

# --- CLI Arguments ---
parser <- ArgumentParser(description = "DNA Distance Benchmark")
parser$add_argument("input_size", type = "integer", nargs = "+", help = "Number of DNA sequences")
parser$add_argument("-u", "--seed", type = "integer", default = 0, help = "RNG seed")
parser$add_argument("-s", "--save", action = "store_true", help = "Save results to JSON")
parser$add_argument("-v", "--verbose", action = "store_true", help = "Debug logging")
parser$add_argument("-c", "--cross", type = "integer", default = 5, help = "Sequence similarity/
args <- parser$parse_args()

# --- Logging Setup ---
log_dir <- Sys.getenv("P_LOGS_DIR", unset = "logs")
if (!dir.exists(log_dir)) dir.create(log_dir, recursive = TRUE)

timestamp <- format(Sys.time(), "%Y%m%d-%H%M%S")
log_file <- file.path(log_dir, glue("bench-dna-{timestamp}.log"))
log_appender(appender_file(log_file))
if (args$verbose) log_threshold(DEBUG) else log_threshold(INFO)

# --- Initialization Logging ---
log_info("Starting DNA benchmark")
log_info("Arguments: {paste(capture.output(print(args)), collapse = ' ')}")
log_info("Log Directory: {normalizePath(log_dir)}")
log_info("System Info: {system('inxi -C', intern = TRUE)}")

set.seed(args$seed)

# --- Helper: Generate Random DNA ---
gen_dna <- function(n, len = 100) {
  bases <- c("A", "C", "G", "T")
  replicate(n, paste(sample(bases, len, replace = TRUE), collapse = ""))
}

# --- Execution ---
cores_total <- parallel::detectCores()
registerDoParallel(cores = 2) # Coarse-grained: 2 trials at once
RcppParallel::setThreadOptions(numThreads = floor(cores_total / 2))

results <- foreach(n = args$input_size, .combine = rbind) %dopar% {
  dna_data <- gen_dna(n)

```

```

start_time <- Sys.time()
res <- compute_dna_metrics(dna_data)
end_time <- Sys.time()

elapsed <- as.numeric(difftime(end_time, start_time, units = "secs"))

log_debug("Processed N={n} in {round(elapsed, 4)}s. Total Div: {res$total_divergence}")

data.frame(n = n, time = elapsed, divergence = res$total_divergence)
}

log_info("Benchmark complete. Summary statistics generated.")

if (args$save) {
  out_path <- file.path(log_dir, glue("results-{timestamp}.json"))
  jsonlite::write_json(results, out_path)
  log_info("Results saved to {out_path}")
}

print(results)

```

4. Concurrency Discussion

Race Conditions and Strategy

In the `DistanceWorker` (`parallelFor`), we avoid race conditions by ensuring each thread writes to a unique memory location in the `RMatrix`. Since the (i, j) and (j, i) indices are unique to the pair handled by the loop iteration i , no two threads ever write to the same cell.

In the `DivergenceReducer`, a naive approach would use a shared `double` and a `std::mutex`. However, mutexes introduce overhead and serialize execution. We use a **functional reduction** strategy:

- Each thread operates on its own `DivergenceReducer` instance.
 - The `join` method is called by the TBB scheduler to merge two local sums once their respective tasks are complete.
 - This is a non-blocking, lock-free pattern that scales linearly with core count.
-

5. Multi-Node Variant Discussion

To scale this to a multi-node cluster, the R test script would transition from `doParallel` to `future` or `pbdMPI`.

- **Control Layer:** Instead of `makeCluster(cores)`, we would use `makeCluster(nodes, type = "PSOCK")` with a list of machine hostnames via `ssh`.
 - **Data Transport:** The DNA sequences (the "input domain") must be serialized and sent to each worker node. For extremely large datasets, we would use a distributed file system or ZMQ to stream chunks of sequences.
 - **Hierarchical Parallelism:** The master node distributes "blocks" of the distance matrix to different nodes. Each node then uses `RcppParallel` locally to compute its assigned block, effectively utilizing all cores across the entire cluster.
-

6. RcppParallel Quick Start Guide

Package Configuration

1. **DESCRIPTION:** Add the following:

- Imports: `RcppParallel`
- LinkingTo: `Rcpp`, `RcppArmadillo`, `RcppParallel`

2. **src/Makevars:** Include the TBB flags:

```
PKG_LIBS = $(shell "${R_HOME}/bin/Rscript" -e "RcppParallel::RcppParallelLibs()")
```

3. **SIMD Optimization:** To enable vectorization, add `CXXFLAGS += -march=native` to your `~/R/Makevars`. R CMD check will warn that this is non-portable; this is expected and acceptable for local performance tuning, but must be removed before submitting to CRAN.

4. **Dependencies:** On Debian/Ubuntu, ensure TBB is available:

- `sudo apt-get install libtbb-dev`

Installation Sequence

Ensure your environment is consistent using `renv`:

```
renv::install(c("Rcpp", "RcppArmadillo", "RcppParallel", "argparse", "logger", "doParallel"))
renv::snapshot()
```

References /g

- [RcppParallel Documentation](#)
- [Intel TBB Design Patterns](#)
- [Hamming Distance in Bioinformatics](#)

Additional Notes /g

- **Thread Pool Management:** Be cautious when nesting `foreach` and `RcppParallel`. If both attempt to use all available cores, the CPU will suffer from excessive context switching (oversubscription). Always use `setThreadOptions` to partition resources.
- **Memory Alignment:** For production DNA tools, consider using `uint64_t` bit-packing to speed up Hamming calculations via XOR and popcount SIMD instructions.

A:1 (ChatGPT)

▣ *Parallel Biological Sequence Comparison in R and C++*

Overview /t

This tutorial introduces *nested parallelism* in R package development using:

- RcppParallel for intra-task shared-memory parallelism (C++)
- foreach + doParallel for inter-task parallelism (R)

The motivating example is:

Pairwise Hamming distance matrix computation for random DNA fragments

This example is ideal because:

- The outer loop over sequence pairs is embarrassingly parallel → parallelFor
- The inner per-sequence character comparison is a reduction → parallelReduce
- The benchmark harness is embarrassingly parallel across trials → foreach

The algorithm naturally expresses a *map-reduce* pattern without artificial API usage.

1. Tutorial Narrative

1.1 Problem Definition

Given:

- N DNA fragments
- each fragment length L

Compute the symmetric N × N matrix:

$$D_{ij} = \sum_{k=1}^L \mathbf{1}(S_i[k] \neq S_j[k])$$

This is:

- CPU-bound
- memory-contiguous
- reduction-heavy

- symmetric

Perfect for:

- `parallelFor` over upper-triangular index space
 - `parallelReduce` inside each pairwise comparison
-

1.2 Why This Fits RcppParallel

- *parallelFor* distributes independent pair computations.
- *parallelReduce* accumulates mismatch counts per character position.
- No shared mutable state.
- No locks required.
- Deterministic output.

This mirrors many bioinformatics kernels used in real pipelines.

1.3 Nested Parallelism Strategy

We separate concerns:

Layer	Parallelism	Tool
C++ kernel	shared-memory threads	RcppParallel
Benchmark trials	process-level	foreach

This enables experimentation with core allocation strategies.

2. C++ Implementation /t

Location:

src/hamming_parallel.cpp

This file uses both `parallelFor` and `parallelReduce`.

```
#include <Rcpp.h>
#include <RcppParallel.h>

using namespace Rcpp;
using namespace RcppParallel;
```

```
// -----
```

```

// Hamming reduction worker (parallelReduce)
// -----
struct HammingReduce : public Worker {
    const RVector<int> x_;
    const RVector<int> y_;
    int sum_;

    HammingReduce(const IntegerVector& x,
                  const IntegerVector& y)
        : x_(x), y_(y), sum_(0) {}

    HammingReduce(const HammingReduce& other, Split)
        : x_(other.x_), y_(other.y_), sum_(0) {}

    void operator()(std::size_t begin,
                   std::size_t end) {
        int local = 0;
        for (std::size_t i = begin; i < end; ++i) {
            local += (x_[i]  $\neq$  y_[i]);
        }
        sum_ += local;
    }

    void join(const HammingReduce& rhs) {
        sum_ += rhs.sum_;
    }
};

// -----
// Distance matrix worker (parallelFor)
// -----
struct DistanceMatrixWorker : public Worker {
    const RMatrix<int> seq_;
    RMatrix<int> dist_;
    const std::size_t n_;
    const std::size_t L_;

    DistanceMatrixWorker(const IntegerMatrix& seq,
                        IntegerMatrix& dist)
        : seq_(seq),
          dist_(dist),
          n_(seq.nrow()),
          L_(seq.ncol()) {}

    void operator()(std::size_t begin,

```

```

        std::size_t end) {

    for (std::size_t idx = begin; idx < end; ++idx) {
        std::size_t i = idx / n_;
        std::size_t j = idx % n_;

        if (j ≤ i)
            continue;

        IntegerVector xi = seq_.row(i);
        IntegerVector xj = seq_.row(j);

        HammingReduce reducer(xi, xj);
        parallelReduce(0, L_, reducer);

        int d = reducer.sum_;
        dist_(i, j) = d;
        dist_(j, i) = d;
    }
}

};

// [[Rcpp::export]]
IntegerMatrix hamming_distance_matrix(IntegerMatrix sequences) {
    std::size_t n = sequences.nrow();

    IntegerMatrix dist(n, n);

    DistanceMatrixWorker worker(sequences, dist);

    parallelFor(0, n * n, worker);

    return dist;
}

```

Header Export

Add:

```

#' @useDynLib dve.sample.r, .registration = TRUE
#' @importFrom Rcpp sourceCpp
NULL

```

Run:

```
Rcpp::compileAttributes()
```

Headers are placed under:

```
inst/include/
```

3. R Test Script (exec/benchmark-hamming.R)

```
#!/usr/bin/env Rscript
```

```
suppressPackageStartupMessages({
  library(optparse)
  library(logger)
  library(glue)
  library(foreach)
  library(doParallel)
  library(RcppParallel)
  library(dve.sample.r)
})
```

```
# -----
# Argument parsing
# -----
option_list <- list(
  make_option(c("-v", "--verbose"), action = "store_true",
              default = FALSE),
  make_option(c("-u", "--seed"), type = "integer",
              default = 0),
  make_option(c("-s", "--save"), action = "store_true",
              default = FALSE),
  make_option(c("-w", "--workers"), type = "integer",
              default = 2),
  make_option(c("-t", "--threads"), type = "integer",
              default = 2)
)
```

```
parser <- OptionParser(option_list = option_list)
args <- parse_args(parser, positional_arguments = TRUE)
```

```
input_sizes <- as.integer(args$args)
```

```
# -----
# Logging
# -----
```

```

log_dir <- Sys.getenv("P_LOGS_DIR", "logs")
dir.create(log_dir, showWarnings = FALSE, recursive = TRUE)

ts <- format(Sys.time(), "%Y%m%d-%H%M%S")
log_file <- file.path(log_dir,
  glue("benchmark-hamming-{ts}.log"))

log_appender(appender_tee(log_file))
log_layout(layout_glue_colors)

log_info("Arguments: {capture.output(str(args))}")
log_info("Log directory: {normalizePath(log_dir)}")
log_info("CPU info: {system('inxi -C', intern = TRUE)}")

set.seed(args$options$seed)

# -----
# Parallel setup
# -----
cl <- parallel::makeCluster(args$options$workers)
doParallel::registerDoParallel(cl)

RcppParallel::setThreadOptions(
  numThreads = args$options$threads
)

# -----
# Benchmark
# -----
results <- foreach(
  n = input_sizes,
  .combine = rbind,
  .packages = "dve.sample.r"
) %dopar% {

  L <- 200L

  seq <- matrix(
    sample(0:3, n * L, replace = TRUE),
    nrow = n
  )

  t0 <- Sys.time()
  d <- hamming_distance_matrix(seq)
  elapsed <- as.numeric(Sys.time() - t0,

```

```

        units = "secs")

data.frame(
  n = n,
  L = L,
  elapsed = elapsed,
  mean_dist = mean(d)
)
}

log_info("Summary:")
print(results)

if (args$options$verbose)
  log_debug("{capture.output(print(results))}")

parallel::stopCluster(cl)

```

4. Concurrency Discussion /t

4.1 Race Conditions

Potential hazards:

- Writing symmetric entries $\text{dist}(i, j)$ and $\text{dist}(j, i)$
- Reduction accumulation

Why Safe?

- Each (i, j) pair computed once
- Symmetric write happens in same thread
- No two threads write same cell
- Reduction uses per-worker copies + join

No mutex required.

4.2 Oversubscription Risk

If:

- workers = 8
- threads = 8

On an 8-core machine $\rightarrow$ 64 threads.

This degrades performance.

4.3 Core Allocation Strategies

Strategy A: Outer Parallelism Only

```
workers = cores  
threads = 1
```

Best when tasks are large.

Strategy B: Inner Parallelism Only

```
workers = 1  
threads = cores
```

Best for single large matrix.

Strategy C: Hybrid

```
workers = 2  
threads = cores / 2
```

Often optimal.

5. Multi-Node Variant (Discussion)

To scale beyond shared memory:

5.1 SSH Clusters

Use:

- `parallel::makeCluster(c("node1","node2"), type="PSOCK")`

Data must be serialized and exported.

5.2 MPI

Use:

- `Rmpi`
- `pbdMPI`

Distance blocks distributed across ranks.

5.3 ZeroMQ (ZMQ)

Use:

- `future`
- `future.batchtools`
- `clustermq`

Controller sends (i, j) blocks as messages.

Message-passing preferred over shared state.

6. RcppParallel Quick Start Guide /t

6.1 DESCRIPTION

Imports:

```
Rcpp,  
RcppParallel
```

LinkingTo:

```
Rcpp,  
RcppParallel
```

6.2 src/Makevars

```
PKG_CXXFLAGS += -DRCPP_PARALLEL_USE_TBB=1  
PKG_LIBS += $(SHLIB_OPENMP_CXXFLAGS)
```

Do not add -march=native here.

6.3 Enabling SIMD Locally

In:

```
~/R/Makevars
```

Add:

```
CXXFLAGS += -march=native -O3
```

CRAN warning during R CMD check is acceptable locally because:

- Not committed
 - Not part of package sources
-

6.4 Install Dependencies (Ubuntu)

```
sudo apt install libtbb-dev
```

6.5 Backend: TinyThread vs TBB

- Default: TBB
 - Fallback: TinyThread (no external dependency)
 - TBB recommended for performance and work-stealing
-

6.6 renv Workflow

```
renv::install("RcppParallel")  
renv::snapshot()
```

7. References /t

- [at-1]: RcppParallel documentation
 - [at-2]: Intel Threading Building Blocks design
 - [at-3]: Hamming distance definition
 - [at-4]: foreach CRAN vignette
 - [at-5]: Writing R Extensions manual
-

Additional Notes /t

- The algorithm is deterministic.
- Suitable for SIMD auto-vectorisation.
- Scales linearly until memory bandwidth saturation.
- Easily extendable to Smith–Waterman using banded DP with block partitioning.
- CRAN compliance preserved.

A:1 (Perplexity)

▣ *RcppParallel* Tutorial

1. Tutorial Narrative /p

This tutorial uses a *Hamming distance matrix* over random DNA fragments as the running example. Each fragment is a short sequence over the alphabet ($\{A, C, G, T\}$), and we want the full pairwise distance matrix for (N) sequences of length (L), where the Hamming distance is the number of mismatched bases at corresponding positions. [pmc.ncbi.nlm.nih](http://pmc.ncbi.nlm.nih.gov)

This is an excellent fit for *RcppParallel* because:

- The distance matrix is embarrassingly parallel across sequence pairs (outer loop).
- Each distance computation is a small, tight inner loop suitable for SIMD and cache-friendly iteration.
- We can naturally express:
 - `parallelFor`: to distribute rows (or blocks of rows) of the distance matrix across threads.
 - `parallelReduce`: to compute global statistics (e.g. sum, min, max, histogram) over all distances without manual locking. [rcppcore.github](https://github.com/rccore/rccore)

In addition, the tutorial shows how to orchestrate multiple benchmark trials in parallel *from R* using `foreach` and `doParallel`, on top of the intra-trial C++ parallelism from *RcppParallel*. [privefl.github](https://github.com/privefl/privefl)

Parallel structure

We adopt the following structure:

- Inner C++ kernel (per trial):
 - `parallelFor`: computes all pairwise Hamming distances between sequences, filling an (N $\times$ N) matrix.
 - `parallelReduce`: traverses the matrix and accumulates global metrics such as:
 - * Total distance sum (for average distance).
 - * Maximum distance.
 - * A simple histogram over distances.
- Outer R harness:
 - Uses `foreach + %dopar%` with `doParallel` to run multiple independent trials (e.g. different input sizes or seeds) in parallel.
 - Each trial calls the exported *Rcpp* function, which internally uses *RcppParallel*.
 - Results (timings and aggregated stats) are combined and optionally saved.

This yields a realistic nested parallel pattern:

- R level: embarrassingly parallel trials across cores or nodes.
- C++ level: shared-memory parallelism with RcppParallel within each trial.

2. C++ Implementation (Rcpp + RcppParallel)

Below is a compact implementation for the core task placed under `src/`, with the corresponding package header generated under `inst/include/`. It uses:

- `parallelFor` to compute the distance matrix.
- `parallelReduce` to compute global summary metrics from the matrix.
- `RcppParallel::RMatrix` for thread-safe access to R matrices. dannyjameswilliams.co

`src/dna_hamming.cpp`

```
// src/dna_hamming.cpp
#include <Rcpp.h>
#include <RcppParallel.h>

using Rcpp::IntegerMatrix;
using Rcpp::List;
using Rcpp::NumericVector;
using Rcpp::StringVector;
using Rcpp::as;
using Rcpp::stop;
using RcppParallel::RMatrix;
using RcppParallel::parallelFor;
using RcppParallel::parallelReduce;
using RcppParallel::Worker;

namespace dve_sample_r {

// Simple utility: map 'A','C','G','T' to small ints (0-3).
inline int EncodeBase(char c) {
    switch (c) {
        case 'A': return 0;
        case 'C': return 1;
        case 'G': return 2;
        case 'T': return 3;
        default: return 0; // fall back (input is random DNA)
    }
}

// Convert character matrix (N x L) to integer codes.
IntegerMatrix EncodeDna(const StringVector& seqs) {
    const int n = seqs.size();
```

```

    if (n == 0) {
        return IntegerMatrix(0, 0);
    }
    const int L = seqs[0].size();
    IntegerMatrix encoded(n, L);
    for (int i = 0; i < n; ++i) {
        const Rcpp::String s = seqs[i];
        if (s.size() != L) {
            stop("All sequences must have equal length.");
        }
        const char* ptr = s.get_cstring();
        for (int j = 0; j < L; ++j) {
            encoded(i, j) = EncodeBase(ptr[j]);
        }
    }
    return encoded;
}

// Worker for parallelFor: compute Hamming distances for matrix rows.
class HammingMatrixWorker : public Worker {
public:
    HammingMatrixWorker(const IntegerMatrix& encoded,
                        RMatrix<int> distances)
        : encoded_(encoded), distances_(distances),
          n_(encoded.nrow()), L_(encoded.ncol()) {}

    void operator()(std::size_t begin, std::size_t end) {
        for (std::size_t i = begin; i < end; ++i) {
            for (int j = 0; j < n_; ++j) {
                int d = 0;
                const int* seq_i = &encoded_(i, 0);
                const int* seq_j = &encoded_(j, 0);
                for (int k = 0; k < L_; ++k) {
                    d += (seq_i[k] != seq_j[k]);
                }
                distances_(i, j) = d;
            }
        }
    }

private:
    const IntegerMatrix& encoded_;
    RMatrix<int> distances_;
    const int n_;
    const int L_;

```

```

};

// Struct for parallelReduce: aggregate distance statistics.
struct DistanceStats : public Worker {
    RMatrix<int> distances;
    std::int64_t sum;
    int max_val;
    std::vector<std::int64_t> hist; // histogram over distance values

    DistanceStats(RMatrix<int> distances, int max_distance)
        : distances(distances),
          sum(0),
          max_val(0),
          hist(static_cast<std::size_t>(max_distance + 1), 0) {}

    DistanceStats(const DistanceStats& other, RcppParallel::Split)
        : distances(other.distances),
          sum(0),
          max_val(0),
          hist(other.hist.size(), 0) {}

    void operator()(std::size_t begin, std::size_t end) {
        const int nrow = distances.nrow();
        const int ncol = distances.ncol();
        const std::size_t total = static_cast<std::size_t>(nrow) * ncol;
        if (end > total) {
            end = total;
        }
        for (std::size_t idx = begin; idx < end; ++idx) {
            const int i = static_cast<int>(idx / ncol);
            const int j = static_cast<int>(idx % ncol);
            const int d = distances(i, j);
            sum += d;
            if (d > max_val) {
                max_val = d;
            }
            if (d ≥ 0 && static_cast<std::size_t>(d) < hist.size()) {
                ++hist[static_cast<std::size_t>(d)];
            }
        }
    }

    void join(const DistanceStats& rhs) {
        sum += rhs.sum;
        if (rhs.max_val > max_val) {

```

```

    max_val = rhs.max_val;
}
const std::size_t m = hist.size();
for (std::size_t i = 0; i < m; ++i) {
    hist[i] += rhs.hist[i];
}
}
};

// [[Rcpp::export]]
List hamming_distance_matrix_parallel(const StringVector& seqs,
                                     const int max_distance = -1) {
    IntegerMatrix encoded = EncodeDna(seqs);
    const int n = encoded.nrow();
    const int L = encoded.ncol();

    if (n == 0 || L == 0) {
        return List::create(
            Rcpp::Named("dist") = IntegerMatrix(0, 0),
            Rcpp::Named("sum") = 0.0,
            Rcpp::Named("max") = 0L,
            Rcpp::Named("mean") = Rcpp::NumericVector::create(NA_REAL),
            Rcpp::Named("hist") = IntegerVector()
        );
    }

    IntegerMatrix dist_mat(n, n);
    RMatrix<int> dist_view(dist_mat);

    HammingMatrixWorker worker(encoded, dist_view);
    parallelFor(0, n, worker);

    const int max_possible = L;
    const int hist_max = (max_distance > 0 && max_distance ≤ max_possible)
        ? max_distance
        : max_possible;
    DistanceStats stats(dist_view, hist_max);
    const std::size_t total = static_cast<std::size_t>(n) * n;
    parallelReduce(static_cast<std::size_t>(0), total, stats);

    const double mean = static_cast<double>(stats.sum) /
        static_cast<double>(total);

    IntegerVector hist_vec(stats.hist.size());
    for (std::size_t i = 0; i < stats.hist.size(); ++i) {

```

```

    hist_vec[static_cast<R_xlen_t>(i)] =
        static_cast<int>(stats.hist[i]);
}

return List::create(
    Rcpp::Named("dist") = dist_mat,
    Rcpp::Named("sum") = static_cast<double>(stats.sum),
    Rcpp::Named("max") = stats.max_val,
    Rcpp::Named("mean") = mean,
    Rcpp::Named("hist") = hist_vec,
    Rcpp::Named("seq_count") = n,
    Rcpp::Named("seq_length") = L
);
}

} // namespace dve_sample_r

```

inst/include/dve-sample-r.h

This header will be generated by roxygen2 from the Rcpp exports, but for clarity, the external signature is:

```

// inst/include/dve-sample-r.h
#ifndef DVE_SAMPLE_R_H_
#define DVE_SAMPLE_R_H_

#include <Rcpp.h>

namespace dve_sample_r {

Rcpp::List hamming_distance_matrix_parallel(
    const Rcpp::StringVector& seqs,
    const int max_distance = -1);

} // namespace dve_sample_r

#endif // DVE_SAMPLE_R_H_

```

In practice, you can ask `Rcpp::compileAttributes()` to generate the .h and the .cpp RcppExports glue. [cran.r-project](https://cran.r-project.org/)

3. R Test Script (exec/)

The driver script:

- Parses CLI arguments via `optparse`.

- Configures logging using `logger` and `glue`.
- Uses `foreach` + `doParallel` to run trials in parallel.
- Calls the C++ function for each trial, measuring elapsed times.
- Demonstrates different core-allocation policies by exposing:
 - Number of `foreach` workers (cluster size).
 - Number of `RcppParallel` threads.
- Optionally saves inputs and outputs per trial.

Assume file: `exec/dna_hamming_benchmark.R`.

```
#!/usr/bin/env Rscript
```

```
suppressPackageStartupMessages(library(optparse))
suppressPackageStartupMessages(library(logger))
suppressPackageStartupMessages(library(glue))
suppressPackageStartupMessages(library(foreach))
suppressPackageStartupMessages(library(doParallel))
suppressPackageStartupMessages(library(jsonlite))
suppressPackageStartupMessages(library(dve.sample.r))
```

```
# ---- CLI argument parsing -----
```

```
option_list <- list(
  make_option(
    c("-v", "--verbose"),
    action = "store_true",
    default = FALSE,
    help = "Enable verbose (debug) logging."
  ),
  make_option(
    c("-u", "--seed"),
    type = "integer",
    default = 0L,
    help = "Random seed for reproducibility."
  ),
  make_option(
    c("-s", "--save"),
    action = "store_true",
    default = FALSE,
    help = "Save input data and results as JSON."
  ),
  make_option(
    c("-n", "--nseq"),
    type = "integer",
    default = 500L,
    help = "Number of DNA sequences per trial."
  )
)
```

```

),
make_option(
  c("-t", "--trials"),
  type = "integer",
  default = 5L,
  help = "Number of benchmark trials."
),
make_option(
  c("-w", "--workers"),
  type = "integer",
  default = 0L,
  help = "Number of foreach workers (0 = detect)."
),
make_option(
  c("-p", "--threads"),
  type = "integer",
  default = 0L,
  help = "Number of RcppParallel threads (0 = default)."
),
make_option(
  c("-c", "--cross"),
  type = "integer",
  default = 0L,
  help = "Number of random crossover events between sequences (0 = none)."
)
)

parser <- OptionParser(
  usage = "%prog [options] input_size...",
  option_list = option_list
)

args <- parse_args(parser, positional_arguments = TRUE)
opts <- args$options
input_sizes <- as.integer(args$args)

if (length(input_sizes) == 0L) {
  print_help(parser)
  quit(status = 1L)
}

# ---- Logging setup -----

script_name <- basename(sub("\\.R$", "", commandArgs(trailingOnly = FALSE)[1L]))
log_dir <- Sys.getenv("P_LOGS_DIR", unset = "logs")

```

```

if (!dir.exists(log_dir)) {
  dir.create(log_dir, recursive = TRUE, showWarnings = FALSE)
}

timestamp <- format(Sys.time(), "%Y%m%d-%H%M%S")
log_file <- file.path(log_dir, glue("{script_name}-{timestamp}.log"))

log_threshold <- if (isTRUE(opts$verbose)) {
  log_threshold(DEBUG)
} else {
  log_threshold(INFO)
}

log_appender(appender_tee(log_file))

test_type <- "dna_hamming"

log_info("Starting {test_type} benchmark.")
log_info("Log directory: {normalizePath(log_dir)}")
log_info("Arguments: {paste(capture.output(str(opts)), collapse = ' ')}")
log_info("Input sizes: {paste(input_sizes, collapse = ', ')}")

inxi_out <- tryCatch(system("inxi -C", intern = TRUE), error = function(e) NA_character_)
log_info("inxi -C output:\n{paste(inxi_out, collapse = '\n')}")

# ---- Parallel backends configuration -----

if (opts$workers ≤ 0L) {
  opts$workers <- max(1L, parallel::detectCores() - 1L)
}

cl <- parallel::makeCluster(opts$workers)
doParallel::registerDoParallel(cl)

if (opts$threads > 0L) {
  RcppParallel::setThreadOptions(numThreads = opts$threads)
}

log_info("Configured {opts$workers} foreach workers, {opts$threads} RcppParallel threads (0 = de

# ---- DNA sequence generation -----

generate_dna_sequences <- function(nseq, length, crossovers, seed) {
  set.seed(seed)
  bases <- c("A", "C", "G", "T")

```

```

mat <- matrix(
  sample(bases, nseq * length, replace = TRUE),
  nrow = nseq,
  ncol = length
)
if (crossovers > 0L && nseq ≥ 2L) {
  for (k in seq_len(crossovers)) {
    i <- sample.int(nseq, 2L)
    cut <- sample.int(length - 1L, 1L)
    tmp <- mat[i[1L], (cut + 1L):length]
    mat[i[1L], (cut + 1L):length] <- mat[i[2L], (cut + 1L):length]
    mat[i[2L], (cut + 1L):length] <- tmp
  }
}
seqs <- apply(mat, 1L, paste0, collapse = "")
seqs
}

```

---- Benchmark core -----

```

run_single_trial <- function(nseq, seqlen, crossovers, base_seed) {
  trial_seed <- base_seed + as.integer(nseq + seqlen + crossovers)
  seqs <- generate_dna_sequences(nseq, seqlen, crossovers, trial_seed)

  t_start <- proc.time()[["elapsed"]]
  res <- dve.sample.r::hamming_distance_matrix_parallel(
    seqs = seqs,
    max_distance = seqlen
  )
  t_elapsed <- proc.time()[["elapsed"]] - t_start

  list(
    nseq = nseq,
    seqlen = seqlen,
    crossovers = crossovers,
    elapsed = t_elapsed,
    sum = res$sum,
    max = res$max,
    mean = res$mean,
    hist = res$hist
  )
}

```

---- Foreach benchmark loop -----

```

results <- foreach(
  i = seq_along(input_sizes),
  .combine = "rbind",
  .packages = c("dve.sample.r")
) %dopar% {
  seqlen <- input_sizes[i]
  trial_results <- vector("list", opts$trials)
  for (k in seq_len(opts$trials)) {
    trial_results[[k]] <- run_single_trial(
      nseq = opts$nseq,
      seqlen = seqlen,
      crossovers = opts$cross,
      base_seed = opts$seed + k
    )
  }
  for (k in seq_along(trial_results)) {
    tr <- trial_results[[k]]
    cat(glue(
      "[worker {Sys.getpid()}] len={seqlen} trial={k} elapsed={tr$elapsed}\n"
    ))
  }
  do.call(
    what = rbind,
    args = lapply(seq_along(trial_results), function(k) {
      tr <- trial_results[[k]]
      data.frame(
        seqlen = tr$seqlen,
        trial = k,
        elapsed = tr$elapsed,
        nseq = tr$nseq,
        crossovers = tr$crossovers,
        sum = tr$sum,
        max = tr$max,
        mean = tr$mean,
        stringsAsFactors = FALSE
      )
    })
  )
}

parallel::stopCluster(cl)

# ---- Aggregation and reporting -----

summary_df <- aggregate(

```

```

elapsed ~ seqlen,
data = results,
FUN = function(x) c(
  mean = mean(x),
  sd = sd(x),
  min = min(x),
  max = max(x)
)
)

summary_df <- do.call(
  rbind,
  lapply(seq_len(nrow(summary_df)), function(i) {
    data.frame(
      seqlen = summary_df$seqlen[i],
      mean = summary_df$elapsed[i, "mean"],
      sd = summary_df$elapsed[i, "sd"],
      min = summary_df$elapsed[i, "min"],
      max = summary_df$elapsed[i, "max"],
      stringsAsFactors = FALSE
    )
  })
)

log_info("Benchmark summary (per length):")
log_info("\n{capture.output(print(summary_df)) > paste(collapse = '\n')}}")

if (isTRUE(opts$verbose)) {
  log_debug("Full results:")
  log_debug("\n{capture.output(str(results)) > paste(collapse = '\n')}}")
}

if (isTRUE(opts$save)) {
  out_prefix <- file.path(
    log_dir,
    glue("{test_type}-{timestamp}")
  )
  results_file <- glue("{out_prefix}-results.json")
  write_json(
    list(
      options = opts,
      input_sizes = input_sizes,
      results = results
    ),
    path = results_file,

```

```

    pretty = TRUE,
    auto_unbox = TRUE
  )
  log_info("Saved benchmark results to: {results_file}")
}

log_info("Completed {test_type} benchmark.")

```

This script satisfies:

- Rscript shebang and location under `exec/`.
- Logging to stdout and file with timestamped name and `P_LOGS_DIR` default. [perceptive-analytics](#)
- `foreach + %dopar%` parallelism for trials.
- Per-trial use of the `RcppParallel` implementation.

4. Concurrency Discussion /p

Natural parallel decomposition

At the algorithm level, each Hamming distance computation between sequence (i) and (j) is independent. We exploit this in two layers:

- `parallelFor` splits the outer index (i) (rows of the distance matrix) into disjoint ranges [begin, end). [rcppcore.github](#)
 - Each worker writes to a distinct stripe of the matrix, so no two threads write to the same element.
 - Reads from the encoded sequences matrix are read-only, so they are safe to share.
- `parallelReduce` iterates over a flattened index space ($0, \dots, N^2 - 1$), aggregating statistics:
 - Each `DistanceStats` object maintains *thread-local* accumulators (`sum`, `max_val`, `hist`).
 - In the `join` method, partial results are combined without contention because joining happens in a controlled, tree-like fashion. [r-bloggers](#)

Avoiding race conditions

Potential race conditions and our strategy:

- Shared write access to the distance matrix:
 - Each thread only writes `distances_(i, j)` for `i` in its assigned range; no overlap between threads.
 - Therefore, no locks or atomics are needed.
- Aggregating global statistics:
 - `parallelReduce` is designed precisely to avoid races by using local copies and a `join` operation. [r-bloggers](#)
 - No explicit mutex is required.
- R objects and memory:
 - We only construct R objects *before* entering parallel regions (`IntegerMatrix encoded`, `IntegerMatrix dist_mat`).

- Inside Worker methods, we use RMatrix views, which are thread-safe accessors to existing R storage. dannyjameswilliams.co

This design follows a functional, message-passing style:

- Each thread computes on its own slice and accumulates local state.
- Only immutable shared data is read concurrently.
- Merging happens explicitly via `join`, avoiding ad hoc locking.

Nested parallelism and oversubscription

The R script may:

- Run several trials in parallel via `foreach` workers.
- Each worker may use multiple `RcppParallel` threads.

This nested parallelism can oversubscribe CPU cores if:

- `workers * threads > physical_cores`.

We mitigate and expose this with configuration options:

- `--workers`: controls the number of R-level worker processes.
- `--threads`: passed to `RcppParallel::setThreadOptions(numThreads = ...)` to bound the C++ thread pool size. cran.r-project

A typical policy on a machine with (C) cores:

- *Outer parallelism dominant*: set `workers = C`, `threads = 1`.
- *Inner parallelism dominant*: set `workers = 1`, `threads = C`.
- *Balanced*: choose `workers * threads ≈ C` (e.g. `workers = 2`, `threads = C / 2`).

The optimal policy depends on relative overheads of R process startup, data generation, and C++ kernel runtime. For CPU-bound kernels with low per-trial overhead, letting `RcppParallel` use most cores and keeping `foreach` sequential often performs best. [privefl.github](https://privefl.github.io)

5. Multi-Node Variant Discussion (No Code)

To extend this benchmark across a multi-node cluster, the main changes are in the R harness:

SSH-based orchestration

Using `parallel::makeCluster` with `type = "PSOCK"` and host names:

- Define a host list with the desired number of nodes (e.g. from a scheduler).
- Use something like:
 - `parallel::makeCluster(c("node1", "node2"), type = "PSOCK", rshcmd = "ssh")`.
- Ensure passwordless SSH and consistent R library paths on all nodes.

- Register the cluster with `doParallel::registerDoParallel(cl)` and keep the C++ code as is.

Data (input sizes, parameters) is small, so passing them as R objects through the PSOCK connections is sufficient. Results are collected back to the master via `foreach`'s `.combine` mechanism. [perceptive-analytics](#)

MPI-based orchestration

Using a package like `doMPI`:

- Replace `doParallel` backend with `doMPI::registerDoMPI`.
- Launch R under `mpirun` or the scheduler's MPI launcher.
- `foreach` workers map to MPI ranks; communication is handled by MPI.

This is more scalable when the number of workers is large, and it can provide better fault tolerance and integration with batch schedulers. The C++ code doesn't change; only the backend and cluster setup differ. [privefl.github](#)

ZeroMQ (ZMQ) control and data transport

For more flexible topologies or dynamic workloads, you can:

- Use packages like `rmq` or higher-level frameworks that wrap ZeroMQ.
- Implement a controller process that:
 - Distributes parameter sets (e.g. `input_size`, `seed`) to workers via PUSH/PULL or REQ/REP sockets.
 - Collects results as workers finish.

This pattern decouples:

- Control flow (the master decides which job goes where).
- Data transport (ZeroMQ sockets over TCP).
- Computation (each worker runs the same R script, calling the `RcppParallel` kernel).

It is especially useful in heterogeneous clusters or cloud environments where nodes join and leave dynamically.

Serialization and data locality

Across nodes, data transfer costs must be considered:

- Inputs: randomly generated DNA sequences can be generated *on the worker* given a seed, so only seeds and parameters need to be transmitted.
- Outputs: only summary statistics and small samples of the distance matrix should be returned; full matrices can be large.

Therefore:

- Keep the Hamming kernel local to each node.
- Use message-passing to send seeds and configuration, not raw matrices.

6. RcppParallel Quick Start Guide /p

DESCRIPTION

Add RcppParallel to LinkingTo and Imports:

Imports:

```
Rcpp,  
RcppParallel
```

LinkingTo:

```
Rcpp,  
RcppArmadillo,  
RcppParallel
```

SystemRequirements: GNU make

If you export C++ functions via Rcpp, also ensure:

Depends:

```
R ( ≥ 4.0)
```

RcppParallel's CRAN page documents the LinkingTo and Imports roles. [cran.r-project](https://cran.r-project.org/web/packages/RcppParallel/index.html)

src/Makevars

You do *not* enable `-march=native` here to keep CRAN-compliant builds portable. A minimal Makevars using the macros exposed by RcppParallel can look like:

```
PKG_CXXFLAGS = $(SHLIB_CXXFLAGS) $(RCPP_PARALLEL_CXXFLAGS)  
PKG_LIBS = $(SHLIB_LIBADD) $(RCPP_PARALLEL_LIBS)
```

These macros are defined by RcppParallel and ensure the correct backend (TinyThread or TBB) is linked. [cran.r-project](https://cran.r-project.org/web/packages/RcppParallel/index.html)

Local SIMD via ~/.R/Makevars

For local development, you can enable SIMD and architecture-specific optimisations with:

```
CXXFLAGS += -O3 -march=native  
CFLAGS += -O3 -march=native
```

Placing this in `~/.R/Makevars` affects local compilation but not CRAN builds, because CRAN compilers ignore user-level flags. You may see a warning similar to “`-march=native` may cause portability issues”; this is acceptable for local benchmarking but must not appear in `src/Makevars` to avoid check failures on CRAN. dannyjameswilliams.co

OS-level dependencies

On Debian/Ubuntu, installing system libraries for RcppParallel and build tools typically requires:

```
sudo apt-get update
sudo apt-get install -y r-base-dev build-essential \
  libtbb-dev
```

RcppParallel can use Intel TBB where available, and libtbb-dev provides the necessary headers and libraries. [cran.r-project](https://cran.r-project.org/web/packages/RcppParallel/index.html)

TinyThread vs. TBB backend

RcppParallel supports two backends:

- TinyThread:
 - Small header-only C++11 thread library bundled with RcppParallel.
 - Used when TBB is not available.
 - Portable but may scale less well on large core counts.
- TBB (Threading Building Blocks):
 - More sophisticated task scheduler.
 - Preferred when libtbb is available and detected at compile time. dannyjameswilliams.co

Selection is automatic based on your build environment:

- If TBB is present, RcppParallel builds against it.
- Otherwise, it falls back to TinyThread.

You can inspect which backend is used via `RcppParallel::defaultNumThreads()` and diagnostic messages in the package documentation. [cran.r-project](https://cran.r-project.org/web/packages/RcppParallel/index.html)

renv sequence

For `dve-sample-r` in explicit mode:

1. Install dependencies into the project library:

```
renv::install(c("Rcpp", "RcppArmadillo", "RcppParallel",
               "optparse", "logger", "glue",
               "foreach", "doParallel", "jsonlite"))
```

2. Snapshot the environment:

```
renv::snapshot(prompt = FALSE)
```

This records exact versions and ensures reproducible builds for the package and the `exec/` benchmark script. [perceptive-analytics](https://perceptive-analytics.github.io)

References /p

- [[^]ap-rcppparallel-doc] RcppParallel CRAN reference manual and website, including `parallelFor`, `parallelReduce`, `RVector`, `RMatrix`, and build configuration notes. [rcpp-core.github](https://rcpp-core.github.io)

- [^ap-parallelrcpp] Articles and blog posts describing basic RcppParallel usage and OpenMP/TBB considerations. [r-bloggers](#)
- [^ap-r-parallelism] Guides to parallelism in R, covering foreach, doParallel, and cluster backends. [privefl.github](#)
- [^ap-swhamming] Literature on parallel sequence alignment, including Smith–Waterman and Hamming-distance-based approaches. [pmc.ncbi.nlm.nih](#)

Additional Notes /p

- For performance profiling, consider using microbenchmark in R for fine-grained timings on individual kernels, and system tools such as perf or vtune at the OS level.
- When scaling the benchmark, monitor memory usage: the distance matrix is ($O(N^2)$) in storage, so large nseq values should be accompanied by summary-only output rather than full matrix retention.

A:1 (DeepSeek)

▣ *Parallel DNA Sequence Comparison with RcppParallel*

Overview /d

This tutorial demonstrates how to harness parallel computing in R for a biologically inspired problem: comparing DNA sequences using a Hamming-based distance matrix. The example naturally maps to two fundamental parallel patterns:

- **Map Pattern** (via `parallelFor`): Computing pairwise distances between sequences
- **Reduce Pattern** (via `parallelReduce`): Aggregating summary statistics across the distance matrix

Why This Example Fits RcppParallel

DNA sequence comparison exhibits **embarrassingly parallel** characteristics:

- Each pairwise distance calculation is independent
- The workload can be evenly distributed across cores
- Both per-element processing (distance computation) and global aggregation (statistics) benefit from parallelization

The RcppParallel package provides thread-safe primitives that integrate seamlessly with R's memory model, making it ideal for this use case.

C++ Implementation

Create `src/dna_comparison.cpp`:

```
// dna_comparison.cpp
#include <Rcpp.h>
#include <RcppParallel.h>
#include <string>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace Rcpp;
using namespace RcppParallel;
```

```
// Helper function to compute Hamming distance between two DNA strings
```

```
int hamming_distance(const std::string& a, const std::string& b) {
    int dist = 0;
    int len = std::min(a.length(), b.length());
    for (int i = 0; i < len; ++i) {
        if (a[i] != b[i]) ++dist;
    }
    dist += std::abs(static_cast<int>(a.length() - b.length()));
    return dist;
}
```

```
// Parallel worker for computing distance matrix rows
```

```
class DistanceMatrixWorker : public Worker {
private:
    const std::vector<std::string>& sequences_;
    RMatrix<int> distances_;

public:
    // Constructor
    DistanceMatrixWorker(const std::vector<std::string>& sequences,
                        IntegerMatrix distances)
        : sequences_(sequences), distances_(distances) {}

    // Process a range of rows
    void operator()(std::size_t begin, std::size_t end) {
        int n = sequences_.size();
        for (std::size_t i = begin; i < end; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == j) {
                    distances_(i, j) = 0;
                } else if (i < j) {
                    int dist = hamming_distance(sequences_[i], sequences_[j]);
                    distances_(i, j) = dist;
                    distances_(j, i) = dist; // Symmetric matrix
                }
            }
        }
    }
};
```

```
// Parallel reducer for computing summary statistics
```

```
class DistanceStatsReducer : public Worker {
private:
    const std::vector<std::string>& sequences_;
    const RMatrix<int>& distances_;
```

```

// Accumulators (per-thread)
double sum_;
double sum_sq_;
int min_;
int max_;
int count_;

public:
// Constructor
DistanceStatsReducer(const std::vector<std::string>& sequences,
                     const IntegerMatrix distances)
: sequences_(sequences), distances_(distances),
  sum_(0), sum_sq_(0),
  min_(std::numeric_limits<int>::max()),
  max_(std::numeric_limits<int>::min()),
  count_(0) {}

// Copy constructor for splitting
DistanceStatsReducer(const DistanceStatsReducer& other, Split)
: sequences_(other.sequences_), distances_(other.distances_),
  sum_(0), sum_sq_(0),
  min_(std::numeric_limits<int>::max()),
  max_(std::numeric_limits<int>::min()),
  count_(0) {}

// Process a range of matrix elements (upper triangle only)
void operator()(std::size_t begin, std::size_t end) {
    int n = sequences_.size();
    for (std::size_t idx = begin; idx < end; ++idx) {
        int i = idx / n;
        int j = idx % n;

        // Only process upper triangle once
        if (i < j) {
            int val = distances_(i, j);
            sum_ += val;
            sum_sq_ += static_cast<double>(val) * val;
            min_ = std::min(min_, val);
            max_ = std::max(max_, val);
            count_++;
        }
    }
}

```

```

// Join results from two threads
void join(const DistanceStatsReducer& rhs) {
    sum_ += rhs.sum_;
    sum_sq_ += rhs.sum_sq_;
    min_ = std::min(min_, rhs.min_);
    max_ = std::max(max_, rhs.max_);
    count_ += rhs.count_;
}

// Extract results
std::vector<double> get_stats() const {
    double mean = (count_ > 0) ? sum_ / count_ : 0;
    double variance = (count_ > 1) ?
        (sum_sq_ - (sum_ * sum_) / count_) / (count_ - 1) : 0;
    double sd = std::sqrt(variance);

    return {mean, sd, static_cast<double>(min_),
            static_cast<double>(max_), static_cast<double>(count_)};
}

};

//' Compute Hamming distance matrix for DNA sequences
//'
//' @param sequences Character vector of DNA sequences
//' @return A list containing distance matrix and summary statistics
//' @export
// [[Rcpp::export]]
List compute_dna_distances(std::vector<std::string> sequences) {
    int n = sequences.size();

    // Allocate distance matrix
    IntegerMatrix distances(n, n);

    // Step 1: Parallel distance computation
    DistanceMatrixWorker worker(sequences, distances);
    parallelFor(0, n, worker);

    // Step 2: Parallel statistics reduction
    DistanceStatsReducer reducer(sequences, distances);
    parallelReduce(0, static_cast<size_t>(n * n), reducer);

    // Extract results
    std::vector<double> stats = reducer.get_stats();

    return List::create(

```

```

    Named("distance_matrix") = distances,
    Named("mean_distance") = stats[0],
    Named("sd_distance") = stats[1],
    Named("min_distance") = stats[2],
    Named("max_distance") = stats[3],
    Named("pairs_count") = stats[4]
  );
}

```

R Test Script

Create exec/benchmark_dna.R:

```
#!/usr/bin/env Rscript
```

```

suppressPackageStartupMessages({
  library(optparse)
  library(logger)
  library(glue)
  library(foreach)
  library(doParallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
  library(RcppParallel)
  library(dveSampleR)
})

```

```
# Parse command line arguments
```

```

option_list <- list(
  make_option(c("-h", "--help"), action = "store_true", default = FALSE,
    help = "Show this help message and exit"),
  make_option(c("-v", "--verbose"), action = "store_true", default = FALSE,
    help = "Enable verbose logging"),
  make_option(c("-u", "--seed"), type = "integer", default = 42,
    help = "Random seed for reproducibility [default: %default]"),
  make_option(c("-s", "--save"), action = "store_true", default = FALSE,
    help = "Save generated data and results to file"),
  make_option(c("-l", "--length"), type = "integer", default = 50,
    help = "Length of DNA sequences [default: %default]"),
  make_option(c("-n", "--nseq"), type = "integer", default = 100,
    help = "Number of sequences to generate [default: %default]"),
  make_option(c("-t", "--trials"), type = "integer", default = 5,
    help = "Number of benchmark trials [default: %default]"),
  make_option(c("-c", "--cores"), type = "integer", default = 4,

```

```

        help = "Number of cores for foreach workers [default: %default]"),
    make_option(c("-p", "--parallel-threads"), type = "integer", default = -1,
        help = "Number of threads for RcppParallel (-1 = auto) [default: %default]")
)

parser <- OptionParser(usage = "%prog [options] input_size",
    option_list = option_list,
    description = "Benchmark parallel DNA sequence comparison")
args <- parse_args(parser, positional_arguments = 1)

if (args$options$help) {
    print_help(parser)
    quit(status = 0)
}

# Setup logging
log_dir <- Sys.getenv("P_LOGS_DIR", "logs")
if (!dir.exists(log_dir)) dir.create(log_dir, recursive = TRUE)

timestamp <- format(Sys.time(), "%Y%m%d-%H%M%S")
script_name <- tools::file_path_sans_ext(basename(args$file))
log_file <- file.path(log_dir, glue("{script_name}-{timestamp}.log"))

log_appender(appender_tee(log_file))
log_threshold(if (args$options$verbose) DEBUG else INFO)

# Log system info
log_info("Starting DNA sequence comparison benchmark")
log_info("Arguments: {paste(names(args$options), args$options, sep = '=', collapse = ', ')}")
log_info("Log directory: {log_dir}")
log_info("Log file: {log_file}")
system_output <- system("inxi -C", intern = TRUE)
log_info("CPU info: {paste(system_output, collapse = ' | ')}")

# Set random seed
set.seed(args$options$seed)

# Generate random DNA sequences
n_seq <- args$options$nseq
seq_length <- args$options$length
input_size <- as.integer(args$args[1])

log_info("Generating {n_seq} DNA sequences of length {seq_length}")
bases <- c("A", "C", "G", "T")
sequences <- replicate(n_seq, {

```

```
paste0(sample(bases, seq_length, replace = TRUE), collapse = "")
})

if (args$options$save) {
  saveRDS(list(sequences = sequences,
              params = args$options),
          file.path(log_dir, glue("input_data-{timestamp}.rds")))
}

# Configure parallel backends
log_info("Configuring parallel backends")
log_info("  Foreach workers: {args$options$cores}")
log_info("  RcppParallel threads: {args$options$parallel_threads}")

# Set RcppParallel threads
if (args$options$parallel_threads > 0) {
  setThreadOptions(numThreads = args$options$parallel_threads)
} else {
  setThreadOptions()
}

# Register parallel backend for foreach
cl <- makeCluster(args$options$cores)
registerDoParallel(cl)
on.exit(stopCluster(cl))

# Benchmark function
benchmark_once <- function(sequences, trial_id) {
  start_time <- Sys.time()
  result <- compute_dna_distances(sequences)
  end_time <- Sys.time()

  tibble(
    trial = trial_id,
    elapsed = as.numeric(difftime(end_time, start_time, units = "secs")),
    mean_dist = result$mean_distance,
    sd_dist = result$sd_distance,
    min_dist = result$min_distance,
    max_dist = result$max_distance,
    pairs = result$pairs_count
  )
}

# Run parallel benchmarks
log_info("Running {args$options$trials} benchmark trials")
```

```

results <- foreach(trial = 1:args$options$trials,
                  .combine = bind_rows,
                  .packages = c("dveSampleR", "dplyr")) %dopar% {
  benchmark_once(sequences, trial)
}

# Aggregate results
summary_stats <- results %>%
  summarise(
    mean_time = mean(elapsed),
    sd_time = sd(elapsed),
    min_time = min(elapsed),
    max_time = max(elapsed),
    mean_distance = mean(mean_dist),
    total_pairs = first(pairs)
  )

log_info("Benchmark complete")
log_info("  Mean computation time: {round(summary_stats$mean_time, 3)}s ± {round(summary_stats$sd_time, 3)}s")
log_info("  Mean pairwise distance: {round(summary_stats$mean_distance, 2)}")
log_info("  Total pairs processed: {summary_stats$total_pairs}")

if (args$options$verbose) {
  log_debug("Detailed results by trial:")
  results %>%
    mutate(across(where(is.numeric), ~round(., 3))) %>%
    as.data.frame() %>%
    capture.output() %>%
    paste(collapse = "\n") %>%
    log_debug()
}

if (args$options$save) {
  # Save results
  saveRDS(list(results = results,
              summary = summary_stats,
              params = args$options),
          file.path(log_dir, glue("benchmark_results-{timestamp}.rds")))
}

# Generate and save plot
p <- results %>%
  ggplot(aes(x = trial, y = elapsed)) +
  geom_point() +

```

```

geom_line() +
geom_hline(yintercept = summary_stats$mean_time,
           linetype = "dashed", color = "red") +
labs(title = "DNA Sequence Comparison Performance",
     subtitle = glue("{n_seq} sequences, {seq_length}bp"),
     x = "Trial", y = "Elapsed Time (s)") +
theme_minimal()

ggsave(file.path(log_dir, glue("performance_plot-{timestamp}.pdf")),
       p, width = 8, height = 6)
}

log_info("Benchmark completed successfully")

```

Concurrency Discussion

Race Conditions and Synchronization

The implementation employs a **functional, data-parallel approach** that minimizes shared state:

1. Distance Matrix Computation (parallelFor):

- Each thread writes to disjoint row ranges of the distance matrix
- No synchronization needed as memory regions are exclusive per thread
- The symmetric matrix update (writing both $[i, j]$ and $[j, i]$) is atomic at the memory level for integer operations

2. Statistics Reduction (parallelReduce):

- Each thread maintains private accumulators (sum_, sum_sq_, etc.)
- Threads only communicate during the final `join()` phase
- No mutexes required as reduction is performed through splitting and joining

3. Potential Race Points:

- The symmetric matrix write could theoretically race if two threads simultaneously write to the same cell, but our row-wise partitioning ensures each cell is written by exactly one thread (when $i < j$)
- Integer writes on x86_64 are atomic, so even if two threads wrote the same cell, it wouldn't corrupt memory (though it would produce incorrect results)

Synchronization Strategy

We deliberately avoided mutexes by:

- Partitioning work by rows/indices rather than using work-stealing
- Using reduction semantics instead of shared accumulators
- Exploiting the symmetric property to compute each pair once

This approach is both **deadlock-free** and **cache-efficient**, as threads work on contiguous memory regions.

Multi-Node Variant Discussion

To distribute the workload across a cluster, several modifications would be needed:

Architecture Options

1. **SSH-based Distribution** (using `parallel::makePSOCKcluster`):

```
# Create cluster across multiple machines
nodes <- c("node1", "node2", "node3")
cl <- makeCluster(nodes, type = "PSOCK",
                  rshopts = c("-o", "StrictHostKeyChecking=no"))
```

- Pros: Simple, uses existing R infrastructure
- Cons: High communication overhead, requires identical R environments

2. **MPI Approach** (using `Rmpi`):

```
library(Rmpi)
mpi.spawn.Rslaves(nslaves = 10)
results <- mpi.parSapply(sequence_batches, compute_distances)
```

- Pros: Efficient for HPC clusters, good load balancing
- Cons: Complex setup, MPI dependencies

3. **ZMQ-based Distribution** (using `pbdr` or `czmq`):

```
# Publisher distributes sequence batches
# Workers compute and return results via ZMQ sockets
```

- Pros: Flexible, language-agnostic, good for heterogeneous environments
- Cons: Requires explicit message handling, error recovery

Work Distribution Strategy

For DNA sequence comparison, we would:

1. Partition the sequence set into overlapping or non-overlapping chunks
2. Distribute chunks to cluster nodes
3. Each node computes its portion using local parallelism (`RcppParallel`)
4. Aggregate results through tree-based reduction

Communication Patterns

- **Control:** Master node coordinates work distribution via SSH/MPI/ZMQ
- **Data:** Sequences broadcast once, results reduced hierarchically
- **Synchronization:** Barrier after computation phase, then reduction

RcppParallel Quick Start Guide

Package Configuration /d

1. DESCRIPTION file additions:

```
Imports: Rcpp (≥ 1.0.0), RcppParallel (≥ 5.1.0)
LinkingTo: Rcpp, RcppParallel
SystemRequirements: GNU make
```

2. src/Makevars (or Makevars.win for Windows):

```
PKG_LIBS = $(LAPACK_LIBS) $(BLAS_LIBS) $(FLIBS)
PKG_CXXFLAGS = -std=c++11
```

3. Enable SIMD locally (optional, for development): Add to ~/.R/Makevars:

```
CXXFLAGS += -march=native
```

Note: This triggers a CRAN check warning but provides ~2x speedup locally. Do not include in package Makevars.

OS Dependencies

Ubuntu/Debian:

```
sudo apt update
sudo apt install build-essential r-base-core r-base-dev
```

R environment setup:

```
# In your project
renv::init()
renv::install(c("Rcpp", "RcppParallel", "RcppArmadillo"))
renv::snapshot()
```

Backend Selection

RcppParallel supports two threading backends:

- **TinyThread** (default): Lightweight, header-only, good portability
- **TBB** (Intel Threading Building Blocks): More sophisticated, better for nested parallelism

To use TBB:

```
# Install TBB system library
# Ubuntu: sudo apt install libtbb-dev

# In R, before loading RcppParallel
Sys.setenv(RCPP_PARALLEL_USE_TBB = "yes")
library(RcppParallel)
```

Testing Setup

```
# Verify configuration
library(RcppParallel)
defaultNumThreads() # Should return number of cores

# Set custom thread count
setThreadOptions(numThreads = 4)
```

References /d

[ad-1]: RcppParallel Documentation. "Parallel Programming with RcppParallel." <https://rcppparallel.github.io>

[ad-2]: Smith, T. F. & Waterman, M. S. (1981). "Identification of Common Molecular Subsequences." Journal of Molecular Biology.

[ad-3]: R Core Team. "Parallel Computing in R." <https://cran.r-project.org/web/views/HighPerformanceComputing.html>

[ad-4]: Intel Corporation. "Intel Threading Building Blocks." <https://www.threadingbuildingblocks.org>

[ad-5]: Weston, S. & Calaway, R. (2021). "Getting Started with doParallel." <https://cran.r-project.org/package=doParallel>

Additional Notes

Performance Considerations

1. **Thread affinity:** Setting thread affinity can improve cache performance on NUMA systems
2. **Memory bandwidth:** The distance matrix scales as $O(n^2)$, so for $n = 5000$, consider block processing
3. **Load imbalance:** Our row-wise partitioning assumes all rows have equal work; for variable-length sequences, dynamic scheduling might be better

CRAN Compliance Tips

- Use `#ifdef _OPENMP` guards for OpenMP-specific code
- Avoid `-march=native` in package Makevars
- Include fallback serial implementations for systems without threading support
- Test on CRAN's strict checking environments using `rhub::check()`

Extending the Example

This pattern can be adapted to:

- Smith-Waterman local alignment (more computationally intensive)

- Phylogenetic distance calculations
- Protein sequence comparison with substitution matrices
- Time series similarity matrices (DTW distance)

A:1 (Mistral)

▣ *Parallelism in R Development: A Tutorial*

Overview /m

This tutorial introduces parallelism in R development with examples in both R and C++. We focus on a practical use case: biological sequence comparison using the Smith-Waterman algorithm on random DNA fragments.

Tutorial Narrative /m

Biological sequence comparison is a fundamental task in bioinformatics. The Smith-Waterman algorithm is a widely used method for local sequence alignment, which is computationally intensive, making it a good candidate for parallelization.

The algorithm involves computing a scoring matrix for two sequences, where each cell depends on the values of neighboring cells. While the computation within a single pair of sequences is inherently sequential, comparing multiple pairs of sequences can be parallelized. This is where RcppParallel comes into play: we can use `parallelFor` to compute the scores for different pairs of sequences in parallel. Additionally, we can use `parallelReduce` to aggregate results, such as finding the maximum score across all pairs.

The test harness uses `foreach` to run benchmark trials in parallel, allowing us to collect performance data for different input sizes and configurations.

C++ Implementation /m

Here is the C++ implementation of the Smith-Waterman algorithm using RcppParallel:

```
#include <Rcpp.h>
#include <RcppParallel.h>
#include <vector>
#include <string>
#include <algorithm>
using namespace Rcpp;
using namespace RcppParallel;

// [[Rcpp::export]]
double smithWaterman(const std::string& seq1, const std::string& seq2) {
```

```

    int m = seq1.size();
    int n = seq2.size();
    std::vector<std::vector<double>> H(m + 1, std::vector<double>(n + 1, 0));
    double max_score = 0;

    for (int i = 1; i ≤ m; i++) {
        for (int j = 1; j ≤ n; j++) {
            double match = (seq1[i - 1] == seq2[j - 1]) ? 1 : -1;
            H[i][j] = std::max({0, H[i - 1][j - 1] + match, H[i - 1][j] - 1, H[i][j - 1] - 1});
            max_score = std::max(max_score, H[i][j]);
        }
    }
    return max_score;
}

// [[Rcpp::export]]
std::vector<double> computeAllPairs(const std::vector<std::string>& sequences) {
    int n = sequences.size();
    std::vector<double> scores(n * (n - 1) / 2);

    parallelFor(0, n, [&](int i) {
        for (int j = i + 1; j < n; j++) {
            int index = i * n + j;
            scores[index] = smithWaterman(sequences[i], sequences[j]);
        }
    });

    return scores;
}

// [[Rcpp::export]]
double computeMaxScore(const std::vector<double>& scores) {
    double max_score;
    parallelReduce(0, scores.size(), [&](int i, double& max_val) {
        if (scores[i] > max_val) {
            max_val = scores[i];
        }
    }, [&](double& max_val1, const double& max_val2) {
        if (max_val2 > max_val1) {
            max_val1 = max_val2;
        }
    }, max_score);
    return max_score;
}

```

R Test Script /m

Here is the R test script that calls the C++ functions and runs benchmarks:

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages(library(argparse))
suppressPackageStartupMessages(library(foreach))
suppressPackageStartupMessages(library(doParallel))
suppressPackageStartupMessages(library(glue))
suppressPackageStartupMessages(library(logger))

# Parse command-line arguments
parser <- ArgumentParser(description = 'Benchmark Smith-Waterman algorithm with RcppParallel')
parser$add_argument('--help', '-h', action = 'store_true', help = 'Print usage and exit')
parser$add_argument('--verbose', '-v', action = 'store_true', help = 'Verbose output')
parser$add_argument('--seed', '-u', type = 'integer', default = 0, help = 'Random seed')
parser$add_argument('--save', '-s', action = 'store_true', help = 'Save data to file')
parser$add_argument('input_size', type = 'integer', nargs = '+', help = 'Input sizes')

args <- parser$parse_args()

if (args$help) {
  print(parser$format_help())
  stop()
}

# Set up logging
log_dir <- Sys.getenv('P_LOGS_DIR', 'logs')
if (!dir.exists(log_dir)) dir.create(log_dir)
log_file <- file.path(log_dir, paste0(basename(sys.frame(0)$ofile), '-', format(Sys.time(), '%Y-%m-%d %H:%M:%S')))
log_appender(appender_file(log_file))
log_layout(layout_simple, date_format = '%Y-%m-%d %H:%M:%S')
log_threshold(if (args$verbose) 'DEBUG' else 'INFO')

log_info('Script arguments: ', paste(names(args), args, collapse = ', '))
log_info('Log directory: ', normalizePath(log_dir))

# Generate random DNA sequences
set.seed(args$seed)
generate_sequences <- function(n, length) {
  bases <- c('A', 'C', 'G', 'T')
  sequences <- replicate(n, paste(sample(bases, length, replace = TRUE), collapse = ''), simplify = 'array')
  return(sequences)
}
```

```

# Benchmark function
benchmark <- function(n, length) {
  sequences <- generate_sequences(n, length)
  scores <- computeAllPairs(sequences)
  max_score <- computeMaxScore(scores)
  list(input_size = n, sequence_length = length, max_score = max_score)
}

# Use foreach to run benchmarks in parallel
cl <- makeCluster(detectCores() - 1)
registerDoParallel(cl)
results <- foreach(i = 1:length(args$input_size), .combine = rbind) %dopar% {
  benchmark(args$input_size[i], 100) # Fixed sequence length for simplicity
}
stopCluster(cl)

# Log results
log_debug('Detailed results: ', results)

```

Concurrency Discussion /m

In the implementation, each pair of sequences is processed independently, so there are no shared resources between threads, and no race conditions occur. The `parallelReduce` function is designed to be thread-safe, ensuring that the reduction operation (finding the maximum score) is performed correctly.

Multi-Node Variant Discussion /m

To distribute work across a multi-node cluster, we can use MPI or ZMQ. For example, with MPI, we could distribute the computation of different pairs of sequences across nodes. Each node would compute scores for a subset of pairs and send the results back to the master node for aggregation. This would involve modifying the R script to use an MPI backend for `foreach` and ensuring that data is properly serialized and transmitted between nodes.

RcppParallel Quick Start Guide /m

1. Package Configuration:

- Add `RcppParallel` to the `DESCRIPTION` file under `LinkingTo`.
- Update `src/Makevars` to include the necessary compiler flags.

2. OS Dependencies:

- On Ubuntu, install the necessary libraries with:

```
sudo apt-get install libtbb-dev
```

3. renv Installation:

- Use `renv::install()` followed by `renv::snapshot()` to manage dependencies.

References /m

1. [RcppParallel Documentation](#)
2. [Smith-Waterman Algorithm](#)

Additional Notes /m

- Ensure that the package passes R CMD check without any warnings (except for the `-march=native` warning in `~/R/Makevars`).
- Follow the `tidyverse` style guide for all R code.
- Use `glue` for log message formatting and `logger` for logging.

This tutorial provides a structured approach to introducing parallelism in R development with practical examples and clear instructions for setting up and running benchmarks.